Catching Misfiling in LLM Tool Narrators with Runtime Contracts Bound to Evidence

Hakan Sabuniş, Yusuf Ünlü, Mehmet Kemal Özdemir

School of Engineering and Natural Sciences, Istanbul Medipol University, Istanbul, Türkiye

hakansabunis@gmail.com, ysffms@gmail.com, mkozdemir@medipol.edu.tr

Extended version: the condensed paper of the same title cites this document for the full audit trail.

Abstract

Large language models (LLMs) increasingly narrate what machine learning tools, such as profilers and leakage detectors, report. A column or number the tool never produced can mislead whoever acts on the explanation. Existing guardrails either constrain decoding, which needs token probabilities commercial endpoints withhold, or check outputs against rules written per application; evaluations report one hallucination rate for every unsupported item. Our key idea is that a deterministic tool fixes every citable entity and number at call time. We therefore propose a runtime contract bound to the evidence: each answer field admits only values from the tool output, and host code checks every answer. We evaluate the contract in the open source assistant **mlcompass** on three LLMs, sixteen leakage analysis tasks, and a profiling task. Without enforcement, the one model that failed often mostly misfiled rather than invented, placing evidence items such as the metric name in inadmissible fields, which the only validated model judge classified as false relations. A field description or the shipped prompt removed most misfiling; allowed values written for other data turned misfiling into invention. Under the contract, no inadmissible column or number reached the user through the checked structured fields, at microseconds of local checking per answer. In a registered test of the best configuration, the check removed residual violations (10 of 600 to none). We contribute admissibility defined per output field, which separates misfiling from invention, an empirical account of how such narrators fail, a comparison of guardrail configurations, and an open replication package.

Keywords: Large language models, faithfulness, runtime verification, structured output, constrained generation, data leakage.

1 Introduction

Machine-learning practice relies on single-purpose tools: profilers (YData, 2026), experiment trackers such as MLflow and TensorBoard (Zaharia et al., 2018; Abadi et al., 2016), and automated model search (Feurer et al., 2015). These tools report numbers, but none of them says what the numbers *mean*. Large language models (LLMs) now fill that gap in many machine-learning workflows: an agent calls the tools, reads their structured output, and explains the results to whoever builds, trains, or ships the model (Yao et al., 2023; Schick et al., 2023; Anthropic, 2024). LLMs, however, produce content that their input does not support (Ji et al., 2023; Huang et al., 2023), and assistants built on LLMs still fail real software tasks (Chen et al., 2021; Jimenez et al., 2024).

A concrete case shows what is at stake. Suppose that a user's regression model scores $R^2 = 1.0$ on held-out data. **mlcompass**, the open-source assistant for machine-learning pipelines that this paper studies, treats a score that high as a sign of data leakage. The tool raises the alarm, for example, when $R^2 > 0.999$, when the area under the receiver operating characteristic curve (AUC) exceeds 0.995, or when accuracy exceeds 0.99 on at least 50 rows. We chose these thresholds ourselves, motivated by the leakage problem that Kaufman et al. (2012) describe. A deterministic detector then measures how strongly each feature correlates with the target and ranks the columns that may be carrying the target into the features. The detector's output is the *evidence*, a dictionary that we write as E. An LLM, the *narrator*, explains E to the user by filling in a structured answer: a verdict chosen from four labels (leakage likely, leakage uncertain, score legitimate, cannot determine), a list of cited columns, and claims of the form (column, statistic, value). On the reference task of this study (Section 5), a sound answer cites the leak column `log_target_v2` with its measured correlation of 0.999.

A structured claim about E can be wrong in two ways. The claim can name an entity or quote a number that appears nowhere in E. The literature calls such an error *extrinsic* (Ji et al., 2023; Maynez et al., 2020), and we call the error *invention*. Alternatively, the claim can place an item that E really contains where E does not admit that item, such as the metric's name `r2` in the list of cited columns, or one column's measurement under another column. This is an *intrinsic* error, which we call *misfiling*. The two kinds of error call for different remedies. Invention calls for grounding the narrator in the evidence, whereas misfiling calls for telling the narrator which field admits what. A single hallucination rate that counts both kinds together can therefore point a tool author to the wrong remedy.

Existing defenses act at three points (Table 1). Constrained decoding restricts the tokens that a model may emit (Willard and Louf, 2023; Beurer-Kellner et al., 2023; Geng et al.,

2023; Lundberg and Ribeiro, 2023; Scholak et al., 2021), but this approach needs the token distribution, which commercial application programming interfaces (APIs) withhold. A provider-side schema (OpenAI, 2024) constrains the structure of each answer, and the caller can check every returned payload against the schema. How the provider produced a conforming payload, and whether the provider will do so for the next request, cannot be observed from outside. Validate-and-reask toolkits, such as Instructor (Liu, 2023) and Guardrails AI (Guardrails AI, 2026), check the returned answer in the caller's code and ask again when a check fails. These toolkits supply the loop, but what each field of the answer may contain is left to the tool author, and a stock installation checks structure only (Section 7).

For the narration of a deterministic tool's output, however, what each field may contain is known when the narrator is called.

Definition 1 (Evidence-closed narration)**.** *A narration task is* evidence-closed *if the set of citable entities and verifiable quantities is finite and machine-enumerable at the moment of the narration call.*

For an evidence-closed task, checking the structured claims against the evidence takes one membership test per field and one numeric comparison per claim, so the soundness of those claims can be enforced rather than merely measured. We build an **evidence-bound runtime contract** on this observation. The narrator answers by calling a tool whose input is described by a JavaScript Object Notation (JSON) Schema, in which an `enum` lists the only values that a field may take. *Tier A* builds these lists from E at call time, so that the column field admits exactly the columns in E. An enum, however, is a request to the model and not a check. *Tier B* therefore rechecks the returned payload in plain code for entity soundness, for value soundness of (column, statistic, value) claims, and for coverage of the top-ranked candidate. Tier B answers a violation with a corrective retry that names the problem. Content that still violates the contract after the retries is stripped, and a persistent omission is flagged. The mechanism is ordinary validate-and-reask, and we do not claim the mechanism as new. What this paper adds is an empirical failure model and an enforceable specification for evidence-closed narration: a field-indexed definition of admissibility, which separates misfiling from invention; measurements of how a narrator fails against that definition; and evidence on how binding the admissible sets at call time compares with rules written in advance. The contract is the default narration path of mlcompass (MIT license), where the contract guards the leakage investigation described above. We describe the contract as shipped rather than as in production, because we have no deployment telemetry. We keep to this narrow scope throughout: a free-text sentence can name only real columns and quote only real values and still assert a causal story that E does not support, and E itself can be wrong.

We ask five research questions (RQs). **RQ1**: How does an unconstrained narrator of evidence-closed material fail, and which changes to the narrator's configuration remove the failure? **RQ2**: Does the contract prevent violations on the channels that the contract checks, and at what cost? **RQ3**: How do the alternatives that a practitioner would reach for compare, namely a stock guardrail toolkit, a provider schema, an allowed-value list written in advance, and a prompt that states the rules? **RQ4**: How far do the wording and the specificity of the instruction move compliance? **RQ5**: Do the contract and the failure carry over to a second kind of evidence?

We answer these questions with 19,132 recorded responses from three models, on sixteen leakage instances drawn from eight sources and on a dataset-profiling task. Without enforcement, `deepseek-chat` fails often and the other two models rarely. On `deepseek-chat`, misfiling rather than invention dominates the violations: under the bare instruction, 1,807 of the 1,829 flagged responses misfile, and 13 use a name that neither E nor the data contains. The one model judge that passed a registered validity check classified every audited case of the dominant misfiling, the metric's name filed as a cited column, as a false relation. A one-sentence description of the cited-column field, or the prompt that mlcompass ships, removes nearly all misfiling; temperature 0 removes most of the misfiling; and the failure returns wherever the fix is absent. An allowed-column list written in advance turns misfiling into invention on data that the list predates, whereas a list that still covers the anchor and part of the evidence does almost as well as the list built at call time. Under the contract, no violation of the checked channels reached the user in any configuration, at microseconds per response. In the best configuration that we found, a registered interleaved test measured what the verifier adds: a violation in 10 of 600 responses without the verifier and in none of 600 with the verifier (Fisher $p = 0.002$; difference 1.7 points, Newcombe 95 % interval 0.7 to 3.0), with no observed reduction in the content measures.

The paper makes four contributions.

1. An empirical account of how a narrator of structured evidence fails. When an evidence-closed narrator fails, misfiling into an under-specified field can dominate invention, and an aggregate hallucination rate hides the difference; on the one model of three that fails often, misfiling dominates. We report contract violations separately from what the violations introduce (Section 6) and draw four design lessons that do not depend on mlcompass (Section 11).

2. A field-indexed definition of admissibility for evidence-closed narration (Definitions 1 and 2). The definition separates misfiling from invention, makes the distinction measurable, and lets a generic membership check enforce soundness. The admissible sets are derived from the producer's output at call time rather than written per application, and the same contract layer serves two evidence shapes (Section 9).

3. A comparison, inside one guardrail toolkit and one provider, of the alternatives that a practitioner would reach for, by what each alternative checks. An allowed-column list written in advance does worse than no list on

data that the list predates. The toolkit's choice semantics, bound to the evidence, match our validator on the field that they check and miss what they do not check. A registered, interleaved test measures what the verifier adds to the best configuration that we found.

4. A complete, auditable replication package (Sabuniş et al., 2026): immutable records of every live response, the preregistration history with its amendments and deviations, the corrected measurements beside the superseded ones, and the registered falsification tests.

Because every number is recomputed from those records, the seventeen faults that we found in our own instruments (Section 10) changed derived measurements and never a record.

We do not claim that LLMs in general misfile rather than invent. The diagnosis rests on one failing model, sixteen leakage instances, and one profiling frame, and two of the three models rarely failed at all. The claim is narrower: when an evidence-closed narrator fails, misfiling can dominate, an aggregate hallucination rate would report the misfiling as invention, and a check bound to the evidence at call time catches both.

2 Background and Related Work

2.1 Closed-world faithfulness and its measurement

Definition 1 is a closed-world assumption (Reiter, 1978) over E: whatever the evidence does not carry is not admissible under the contract. The setting is old. Data-to-text and meaning-representation-to-text generation narrate a finite record and measure faithfulness against that record offline, using the slot error rate over the input's slots (Dušek et al., 2020), relation-generation precision against the source table (Wiseman et al., 2017), or table-grounded scoring (Dhingra et al., 2019). In the taxonomy of Ji et al. (2023) and of Maynez et al. (2020), a name absent from E is an *extrinsic* error, and a real evidence item asserted in the wrong place is an *intrinsic* one. We use *invented* and *misfiled* for these two classes on a structured channel and claim neither category as new. Xu (2025) argues that hallucination is unavoidable in an open world but can be mitigated in a closed one. Luo et al. (2026) repair a model's own explanations by removal, and AVA (Karnatak et al., 2026) verifies citations against a curated report corpus and abstains when the corpus cannot support an answer. For an evidence-closed tool narrator, what changes is where the check runs: at call time, on the payload that the user will see, rather than offline on a corpus.

Tool-use frameworks (Yao et al., 2023; Schick et al., 2023; Anthropic, 2024) govern how an agent acts, but they say nothing about what the agent may afterwards claim about a tool's output. Provenance work reconstructs evidence support from traces after the fact (Wang et al., 2026), and Ng et al. (2026) argue that agent safety belongs in a runtime contract. Static leakage detectors for notebook code address the leakage problem itself before training (Yang et al., 2022; Truong et al., 2025); our narrator explains a detector's findings after training. The enforcement mechanism is classical. Tier B is a postcondition in the sense of Design by Contract (Meyer, 1992), and deleting what violates the postcondition is *suppression*, the weakest edit that an edit automaton can make (Ligatti et al., 2005). Schneider (2000) and Falcone et al. (2012) characterize which properties a monitor can enforce at all.

2.2 Where enforcement acts, and when the allowed set is fixed

Table 1: Ways to constrain what an LLM may output. *Locus*: where the constraint acts: while the model picks tokens (decoding), in the provider's handling of a schema, or in code that checks the returned answer. *Bound*: when the allowed set is fixed, when the tool is written (author) or for each request from the data (call). *Logits*: whether the method needs the model's token probabilities, which commercial APIs do not expose. *Guarantees*: what holds for every output. This paper's row shares its locus and binding time with validate-and-reask tools; the differences are the field-indexed domain derived from a deterministic producer, and what we measure.

Locus	Bound	Logits	Guarantees	Example
Decode, grammar	author/call	yes	grammar	Willard and Louf, 2023; Geng et al., 2023; Scholak et al., 2021; Poesia et al., 2022
Decode, span	call	yes	verbatim spans	Signé et al., 2026
Decode, closed loop	call	yes	none (empirical)	Cruz, 2026
Provider schema	author/call	no	structure, as a service guarantee	OpenAI, 2024
Code, policy	author	no	policy contract	Ahn and Kim, 2026; Zhang et al., 2025b; Winston et al., 2026
Code, context + reask	call	no	what the validator checks	Liu, 2023; Guardrails AI, 2026
Code, ledger	accumulating	no	per-claim provenance	Du et al., 2026
Code, field-indexed	call	no	(C1)–(C2), (C3) flagged	this paper

Decode-time methods can also bind constraints to the input. Grammar-constrained decoding admits input-dependent grammars (Geng et al., 2023); PICARD and Synchromesh, constrained decoders for database queries and code, check partial programs against a database schema or a program's scope (Scholak et al., 2021; Poesia et al., 2022); monitor-guided decoding consults a static analysis of the repository (Agrawal et al., 2023); and CHyD (Signé et al., 2026) restricts retrieval-augmented output to spans of the retrieved context. All of these methods need the token distribution, which a commercial endpoint withholds. ATLAS-RTC (Cruz, 2026), a token-level runtime controller, intervenes during decoding with masking and rollback and reports empirical gains without a guarantee. Le (2026) shows that the wording of schema keys moves accuracy in both directions, and JSONSchemaBench (Geng et al., 2025) finds that constrained-decoding frameworks differ in which schema features they cover, so what a schema enforces depends on the engine that applies the schema. A valid schema does not make an answer right either: on OrderBench (Li, 2026), the strongest of four ordering agents produces schema-valid output every time and still gets about one order in five wrong.

Other tools already occupy this paper's row of Table 1. Instructor (Liu, 2023), a Python library for structured LLM output, validates a response against context supplied at call time and re-asks on failure. Guardrails AI (Guardrails AI, 2026), an open-source validation framework, and NVIDIA's NeMo Guardrails (Rebedea et al., 2023) ship validate-and-reask loops whose validators can read call-time metadata. Our

mechanism is one instance of that practice, and we add three things to the practice. The first is the field-indexed admissible domain, derived from a deterministic producer's output, which separates misfiling from invention and which a generic membership check can enforce. The second is a set of measurements: which failure actually occurs, what enforcement costs, and which alternatives reach the same result. The third is a shipped artifact whose narration path is the one that we measured.

Three pieces of work are closest. Ahn and Kim (2026) vary the enforcement layer with the model fixed, over 270 runs on three hosted models, and find what we find. Prompts let violations through, a bolt-on guardrail over-refuses (88/120 utility against 120/120), and code-owned enforcement keeps both safety and utility. Their contract is a policy fixed when the harness is written, whereas ours has no admissible domain until the evidence exists. The same difference separates our contract from RvLLM (Zhang et al., 2025b), which checks outputs at runtime against constraints that domain experts write in a specification language. The difference also separates our contract from the work of Winston et al. (2026), who translate natural-language policies into satisfiability-modulo-theories (SMT) constraints and check each tool call before the call runs. LedgerMind (Du et al., 2026) lets claims cite only active ledger entries, checks entity and numeric grounding, and restricts repair to transitions that add no unprovenanced content. LedgerMind thus covers three of our channels and our repair asymmetry (Observation 1), which we derived independently. LedgerMind's ledger grows during a trajectory, whereas ours is complete before the call. EviBound (Chen, 2025), which appeared eleven months before our measurements, gates a machine-learning research agent by querying MLflow for a run, its artifacts, and a terminal status, with bounded retries, and checks metric values when its acceptance criteria name them. EviBound verifies that what an agent claims to have done and achieved meets criteria written for the task. We instead derive, from a fixed tool output, which entity and which quantity each field of a narration may hold, and we measure how narrators fail that binding. Under that binding, the narrator may report `log_target_v2`'s correlation as 0.999, but may not list `r2` as a column or quote a correlation that E does not contain.

3 Evidence-Closed Narration

3.1 The evidence, and why admissibility is per field

In the leakage investigation, the evidence dictionary E holds four things: the suspicious metric and its value; for the ten features most correlated with the target, the correlation of each feature with the target y, taking whichever of ρ_P and ρ_S has the larger magnitude and keeping its sign; a ranked candidate-leak set $C = \{c : \max(|\rho_P(c, y)|, |\rho_S(c, y)|) \geq 0.99\}$; and the perfect-match rate $P(\hat{y} = y)$. We report Spearman's coefficient because a monotone transform of the target pulls ρ_P down while ρ_S stays at 1. Our log-target leak has $\rho_P = 0.93$ and $\rho_S = 0.999$, so a filter that used Pearson's coefficient alone would miss the leak.

Let N_E be every name that E carries: its keys and its string values. The obvious formalization admits any member of N_E in any reference field. Under that formalization, a column field reading `r2` counts as sound, because `r2` is in E as the name of the *metric*. We therefore index admissibility by field.

3.2 The contract, stated as an invariant

Let F be the answer's entity fields: the list of cited columns and the column of each claim (`columns_referenced` and `claims[].column` in the schema). Their entries, the *leaves*, are names compared as strings. E determines the admissible column set $A_{E,\text{col}}$ and a value table V_E keyed by (column, statistic). The statistic names that V_E uses form $A_{E,\text{stat}}$, which Tier A also binds into the schema's statistic field, so $\text{dom}(V_E) \subseteq A_{E,\text{col}} \times A_{E,\text{stat}}$. A statistic outside $A_{E,\text{stat}}$ is therefore a pair that E does not carry, which condition (C2) below rejects. Column and statistic sets may share a name, as when a frame has a column called `mean`; each name is checked in its own field.

A leaf outside $A_{E,\text{col}}$ is *misfiled* when the leaf lies in N_E. Otherwise, the leaf is outside the evidence, and for measurement we split that case by the data. A real column of the frame that E does not list is *unlisted*, and a name that exists in neither E nor the frame is *invented*. The verifier sees only E and treats all three cases alike. A response *commits* when its verdict is not empty and is not the abstention "cannot determine". The entities that a response *references* are its leaves in the column fields.

Definition 2 (Evidence-bound runtime contract)**.** *A structured response r* satisfies *C_E, with tolerance τ and anchor a^*, if **(C1, entity soundness)** every leaf of r in a field of F lies in $A_{E,\text{col}}$; **(C2, value soundness)** every claim (e, s, v) in r with $e \in A_{E,\text{col}}$ has $(e, s) \in \text{dom}(V_E)$, v a finite number, and $|v - V_E(e, s)| \leq \tau$; and **(C3, anchor coverage)** if r commits, the anchor a^*, the top-ranked candidate, is among the entities r references.*

Conditions (C1) and (C2) together state *referential and value soundness with respect to E* on the structured channels. This property is narrower than faithfulness in general: the verdict, the free-text narration, and the suggested checks lie outside the property, and E itself is trusted. Tier B enforces (C1) and (C2). A response that violates either condition is sent back, at most twice, with the violation named, and whatever still violates the contract after that is deleted. A response with no admissible verdict is retried in the same way and, if the problem persists, aborted rather than completed on the narrator's behalf. Hence *no structured claim that Tier B returns violates (C1) or (C2)*; this follows from the deletion step (Observation 1).

Whether our Python implementation realizes this algorithm is a separate, empirical question. Thirty-six hand-written mutation tests and a property-based search over 10^6 generated

(evidence, response) pairs support the implementation without proving the implementation correct. The search was registered in the analysis plan before the study but built after the study, and the search found two defects in an earlier version (Section 10). The search draws evidence with 1 to 12 columns and 1 to 5 statistics. The responses that the search generates use names that are admissible, taken from another field, or random, and values that sit inside or just outside τ or are non-finite, non-numeric, missing, or a 400-digit integer. Each mutation test disables one check; one of these tests covers a case in which stripping creates an omission. Ten fault-injection tests then drive the shipped path end to end with a narrator that repeats one fault on every attempt: an inadmissible cited or claimed column; a wrong, non-finite, or 400-digit number; an omission; an inadmissible verdict; or a missing tool call. The tests check the delivered payload, its flags, and the text that the panel prints, and six of them fail when the deletion step is disabled. Stripping, the omission flag, the abort, and the post-strip re-check are therefore exercised on the delivery path, although no live response reached them. Deletion removes whole claims, so a delivered claim never lacks a field.

Because the (C1)–(C2) guarantee does not come from sampling, we report an enforced arm's clean result on those conditions as what the verifier did, and we attach no interval to that result. The (C3) result is an observation like any other. Condition (C3) cannot be enforced in the same way, so the contract reports (C3) but does not guarantee the condition: a committed response that still omits the anchor after the retries reaches the user with the omission flagged. When E has no candidate, there is no anchor, and (C3) holds vacuously.

Observation 1 (Repair asymmetry)**.** *Soundness is safe-repairable by deletion: the projection $\Pi_E(r)$ removing every violating element is sound, idempotent, and the identity on sound responses. Anchor coverage is not: any repair must add a reference the model did not produce.*

Proof sketch. Deletion preserves membership and tolerance and can remove every violating element. For anchor coverage, deleting the verdict would discharge the obligation, but the schema requires a verdict, and substituting "cannot determine" would attribute to the narrator a stance that the narrator did not take. Both premises are stipulations of this contract, and the artifact honors them by aborting rather than substituting. Given these premises, every remaining repair inserts an anchor reference that is absent from r. □

Tier B therefore strips unsound content and only *flags* a persistent omission, which in edit-automaton terms is suppression without insertion (Ligatti et al., 2005). In the security setting, insertion is simply a stronger primitive; applied to a narrator, an inserted reference would fabricate attribution. With each $A_{E,f}$ and V_E hashed once, checking a response costs $O(|r|)$. The step that needed care was keying V_E on the pair, which our own scorer once failed to do (Section 10).

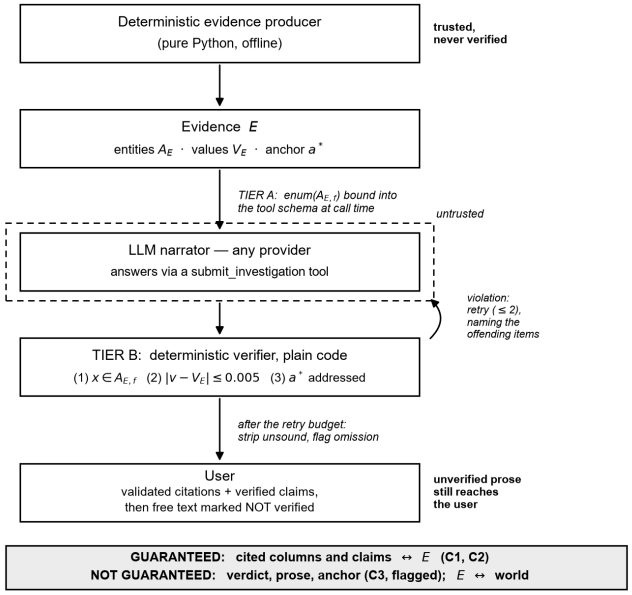


Figure 1: The evidence-bound contract. Tier A binds the tool schema's enums to E at call time. Tier B re-checks the returned answer in host code, retries with the offending items named, and after the budget strips unsound content and flags a persistent omission. The dashed boundary encloses only the narrator. The producer is trusted and never verified, the free-text field reaches the user marked unverified, and the guarantee is consistency with E, which says nothing about whether E matches the world.

4 The Artifact

4.1 Architecture and the trust boundary

Figure 1 splits the system at a trust boundary. One side holds everything deterministic: the pure-Python evidence producer, E, the Tier B verifier, and the user-facing output. The other side holds the LLM narrator, which we treat as untrusted whoever serves the model. The system has three *layers*: evidence production; a narrator told the rules in words (Section 7 measures what that is worth); and the evidence-bound schema, through whose submission tool the answers arrive. The contract spans the last two layers as two enforcement *tiers*. Tier A restricts the cited-column list and each claim's column to enums built from E at call time, and an enum remains a request to the model. On `deepseek-chat`, the contract's Tier A arms ran without the provider's strict decoding flag, whereas the typed-schema arms, `A-STRICT-ENUM`, and the lists of A14 sent the flag. The harness sent the flag to the provider's standard endpoint, whose documentation does not describe strict mode, and the records show that the flag was not enforced: on the crowded instance, 18 of the 200 responses of the author-time list named the anchor, which the list does not contain, 5 of them beside invented names from the list. No arm therefore measured decoding enforced by the provider. Tier B re-checks the returned tool input in plain code for (C1), for (C2) within $\tau = 0.005$, and for (C3).

Algorithm 1 gives the control flow. Anchor coverage is

Algorithm 1 Evidence-bound narration (budget $k = 2$)

$B \leftarrow \text{bind}(E)$ ▷ $A_{E,f}$, V_E, τ, a^*
$S \leftarrow$ schema with enums from B ▷ Tier A
$c \leftarrow$ empty correction
for $i = 0, \ldots, k$ **do**
  $r \leftarrow \text{model}(\text{prompt} + c, S)$ ▷ provider errors propagate
  if r has no admissible verdict **then**
    $c \leftarrow$ fixed correction; **continue**
  end if
  $viol \leftarrow \text{verify}(r, B)$ ▷ (C1), (C2), (C3)
  if $viol = \emptyset$ **then break**
  end if
  $c \leftarrow$ correction naming $viol$
end for
if r has no admissible verdict **then return** abort ▷ evidence panel only
end if
$r \leftarrow \text{strip}(r, B)$ ▷ delete leaves violating (C1), (C2)
$flag \leftarrow r$ commits $\wedge\ a^* \neq \bot \wedge a^* \notin \text{refs}(r)$
return $(r, flag)$ ▷ (C3) flagged, never repaired

checked after stripping, because deleting an unsound claim can orphan the anchor. A response with no admissible verdict (no tool call, unparsable arguments, or a verdict outside the enum) is aborted after the budget, and no verdict, claim, or prose is shown. The deterministic evidence panel is printed in every case and lists the candidate leak columns with the anchor first, so neither an abort nor a flagged omission hides the anchor. The abort path is recent: the earlier code passed such a response as an empty answer. In the records, such responses occur only on arms without a verifier (5 on `deepseek-chat` and 8 on `qwen2.5:7b`), so no reported number depends on the change.

4.2 One contract, two instances

For most of this study, the code handled a single instance: one column set, each claim's statistic fixed to correlation, and each claim checked by looking up its column in the correlation map. The contract is now a layer. A specification says how to read one evidence shape; binding yields the field-indexed domains, the value table, and the anchor; and the verifier, the stripper, the retry text, and the schema builder name no task. Leakage is one specification of about forty lines. A dataset profiler is a second: every column of the frame is admissible, each column has up to thirteen statistics, and the statistic set is computed per call, because a categorical column carries a cardinality and no mean. Each specification also names its anchor: the top-ranked candidate for leakage, and the column with the most missing data for the profile. The profile narrator runs in the measurement harness, and the leakage narrator is the one that the product ships.

Extracting the layer exposed something that the single task could not: the value table must be keyed on (entity, statistic). With one statistic per column, keying on the column and keying on the pair coincide. In a profile, however, the column `age` has mean 38.5 and missing fraction 0.012, and a table keyed on the column alone checks a claim about one of these quantities against the other. That error is one of the scorer defects in Section 10, reappearing in the production path. The evidence's surface shape also has to match the flat claim slot: handed a nested quantity, the narrator in our runs invented a flattening, five different ones in all.

4.3 What the invariant does not cover, and a threat model

Four things fall outside the invariant. The first is the verdict and the prose. The verdict is checked for admissibility but not for agreement with E, so a narrator may cite the planted leak and still answer "score legitimate". None did, and on the negative control no response says "leakage likely" (Appendix A.4). The free-text narration and the suggested checks are printed beneath the validated content and marked unverified; Section 12 measures what they carry.

The second is the correctness of E. If the leakage detector mis-ranks a candidate, the contract faithfully narrates a wrong finding. The producer is trusted and not verified, and (C3) names one anchor rather than every finding. A response that addresses the second of two nearly tied candidates would be flagged; no instance that we ran has such a tie.

The third is the other narration surfaces. The product prints two other narrations of the same evidence that no contract governs: an optional LLM interpretation panel printed by the evaluation command, and a tool-server command whose calling model narrates in the user's editor (Anthropic, 2024). The `advise` command also sends the dataset profile to an LLM without the contract; the profile narrator of Section 9 runs only in the measurement harness. The interpretation panel is marked unverified. The tool-server narration carries no mlcompass marker at all, because the client model writes that narration. mlcompass guarantees the contract panel and nothing else.

The fourth is adversarial evidence. Column names come from the user's data and reach the schema's enums, the prompt, the corrective retry, and the rendered panel, and membership in E does not make a string safe. A crafted header is a prompt-injection vector (Greshake et al., 2023). Schema keys and enums are part of the model's instructions (Le, 2026), and through Tier A's enum values a header reaches the channel that the EnumAttack of Zhang et al. (2025a) uses to carry a harmful request past a model's safety training. The contract guarantees only that what is shown is in E, and rendered strings are escaped. Capping name length, restricting names to an identifier-like character set, and quoting names as data would each reject or alter some legitimate headers; we have neither implemented nor measured these defenses.

5 Study Design

Following the ACM SIGSOFT Empirical Standards (Ralph et al., 2020), the community's checklists for what an empirical software-engineering study should report, we designed the study as a controlled experiment over fixed evidence dictionaries. The manipulated variable is the enforcement mechanism (RQ1–RQ3, RQ5), the instruction wording (RQ4), or the naming and description of the evidence (the controls of RQ1 and

RQ5). The analysis plan and its amendments were frozen before the runs that they govern and are in the replication package. Table 2 reports what the plan registered and what became of each item, and Table A1 in Appendix A completes the list. The plan is a pre-registration kept in the replication package with its commit history; the plan is not a Registered Report accepted by a journal. We call a result *confirmatory* only when the plan fixed its prediction and test before the runs: the X- and H-items of Table 2, the prediction R2a on the twelve frozen instances (A3), the interleaved naming run (A9.1), the leakage retry audit (A9.2), the interleaved test of the verifier (A11), the interleaved test of partly stale lists (A14), and the validity check of each audit judge (A10, A12, A13). Every other comparison, including every comparison across dates or harness versions, is exploratory.

5.1 Subjects

The subject is the shipped leakage-investigation path. The *reference* task is a 1,000-row frame of fourteen columns: the target `y_true`, the prediction, and twelve candidate features. The target is $y \sim \mathcal{N}(50, 10^2)$. The frame carries a monotone transformed leak, `log_target_v2` $= \log(y - \min y + 1) + \varepsilon$ with $\varepsilon \sim \mathcal{N}(0, 0.01^2)$, which is the kind of column left behind when a log-transformed copy of the target stays among the features, and a high-correlation distractor. The evidence lists ten columns and has the anchor `log_target_v2`. A second, *crowded* instance has 1,200 rows, an anchor `sensor_ref` that correlates at 0.995 with the target, and nine near-tied distractors, `sensor_00` to `sensor_08`, at 0.900–0.924. A column list written against the reference task covers none of the crowded instance's ten evidence columns.

Two further instances use real data, each narrated after an ordinary least-squares fit on all numeric features. *Bodyfat*, with 252 men (Penrose et al., 1985), has a documented leak: body fat is computed from body density by Siri's equation, so the fit reaches $R^2 = 0.978$, and the detector names `Density` ($|\rho| = 0.993$) as the only candidate. *Sambanis* is the civil-war onset data of Hegre and Sambanis as distributed by Muchlinski et al. (2016) (7,140 country-years). The fit reaches $R^2 = 0.204$, and no column is a candidate, so this instance is a negative control with no anchor. Neither score reaches the product's trigger ($R^2 > 0.999$), so on both datasets we call the narrator directly. The prompts, schemas, and contract were written against the reference task; the crowded instance and the two datasets played no part in their design.

Twelve more instances were frozen in the plan on 2026-07-07, before any run of the registered campaign, and were first run for this revision. Each instance plants one of six registered target-derived leaks (monotone transform, noisy proxy, exact copy, inverse, binned copy, duplicated rows) in one of four public tables, so that each leak appears in two tables. The tables are insurance charges (Lantz, 2013) (simulated from census figures), Cleveland heart disease (Janosi et al., 1988), IBM telco churn (IBM, 2019), and Ames housing (De Cock, 2011). In every arm, the shipped leakage detector produces E. Section 9 describes the profile task of RQ5.

5.2 Arms

An *arm* is one experimental condition: a fixed system prompt, tool schema, and check on the returned answer. Table 3 lists the arms. Their names follow the analysis plan and the replication package, so every number can be traced to its records. The *bare* prompt states the task and no faithfulness rule. The *shipped* prompt, the one that mlcompass uses, states seven rules (among them: cite only items in E, copy numbers exactly, and address the top candidate) and a verdict mapping. The shipped prompt also tells the model that the schema restricts the column field and that violations are rejected; on the arms without enforcement, neither statement is true. An *open* schema has the answer's fields and lists no admissible values.

Several arms isolate one mechanism. `A-CONTRACT` runs the shipped enforcement under the bare prompt, so that the mechanism is not confounded with the instruction. `A-STRESS` removes the Tier A enums, so that every Tier B catch is observable. `A-GR-CHOICES` is our reimplementation of the toolkit's acceptable-values validator, with its list built from E at call time, because a fresh install of Guardrails AI 0.11 registers no validator. `A-GR-CHOICES` is therefore not the library's own implementation, and we did not test the arm against the validator that the Guardrails Hub offers. Every arm with a retry loop has a budget of 2. Tier B and `A-GR-OURS` check (C1) and (C2) and flag (C3); `A-GR-CHOICES` checks (C1) on the cited list only; and `A-GR-STOCK` checks structure only. The comparisons therefore measure what each configuration checks, not what each library can be made to check. `A-GR-STOCK` and `A-STATIC-ENUM-STALE` are controls rather than competitors of equal strength. They are what a practitioner gets by default: a toolkit's stock validators, which check structure, and a column list written once while authoring. The competitor of equal strength is `A-GR-OURS`, which runs the same checks as Tier B inside the toolkit's loop.

After review, we added controls. The column description of `A-L1-DESCRIBED` reads "Names of dataset columns (features) from the evidence that you cite. Column names only: not metric names such as r2, and not statistic names." That sentence names the failing token, so the `-NEUTRAL` arms end the description instead with "Column names only, not the names of metrics or statistics." `A-L2-RULES` gives the bare prompt followed by the shipped prompt's first three rules, with no claim that anything is checked. `A-L2-NOCLAIMS` is the shipped prompt with its two sentences claiming that violations are rejected removed and nothing else changed, and `A-L1-T0` repeats the floor at temperature 0. `A-STRONG-NOVERIFY` and `A-STRONG-CONTRACT` give the narrator everything that the study found to help (the shipped prompt, the call-time enum, and the neutral field description) and differ only in whether the payload is verified. These controls are not part of the registered plan and are exploratory. The confirmatory tests that we then registered (amendment A3) were cut short by the provider budget (A4). The controls ran between 2026-09-24 and 2026-09-30 on later harness commits, which add the options, and each record pins its commit. The baseline arms were also run on the crowded instance, where the author-time

Table 2: What the analysis plan predicted, and what happened. X-items are planned comparisons between two arms; X1, X3, X4, and X5 form the Holm-corrected family. H-items are hypotheses, numbered as in the plan (H1–H6 belonged to an earlier round). Differences are in percentage points (pp) of the rate of responses reaching the user with any violation, on the 2026-09-15 runs. Intervals are Newcombe 95 % intervals, and Fisher tests are Holm-adjusted over the three family comparisons that ran (X5 was not run; counting X5 as $p = 1$ would move X3's adjusted p from 0.015 to 0.022). X- and H-items were committed on 2026-09-14 at 19:34 UTC, and the first record that they govern started on 2026-09-15 at 18:23 UTC. A3 was committed on 2026-09-30 at 15:50 UTC, and its first call started at 15:51 UTC. Every record pins the harness commit under which the record ran. The plan registered a 2-point equivalence band, and claims superiority only where the interval excludes 0 and the point difference reaches 5 points; a significant difference below 5 points is reported as significant and not material. X3's difference is significant and, on its point difference, not material, although its interval reaches 7.7 points; X4's arms both verify, so their tie holds by construction. *Status*: as planned, deviated (run with the change that the row describes), exploratory, not tested, or not run. The five rows after the first rule record where the study departed from the plan; the rows after the second rule are the amendments, registered for this revision, on which the results rest. Table A1 in Appendix A lists the others (A3 R3, A3 R4, A7, A9.2, A10, A12).

Item	Prediction	Outcome	Status
X1	Contract below the stock Guardrails loop	Stock loop higher by 35.5 pp [28.9, 42.3], $p_{\text{Holm}} < 10^{-23}$; registered on a real-data task, where only the floor, shipped, and strong arms were later run; reported on the synthetic task	deviated
X2	Contract below the strict call-time enum on a channel other than entity	Void: no such channel activated on leakage	not tested
X3	Contract below a typed schema without column list	Schema higher by 4.0 pp [1.3, 7.7], $p_{\text{Holm}} = 0.015$, not material on the point difference (interval to 7.7); that arm sent the strict flag, which was not enforced	deviated
X3b	Contract against the author-time list (exploratory)	Equal on the reference task; 76.0 against 0 % on the crowded instance (invented names 72.5 %); A1's prediction, between no list (27.5 %) and the call-time list, rejected	exploratory
X4	Our checks inside the Guardrails loop tie the contract	0.0 pp [−1.9, 1.9], $p = 1$; both arms verify, so the tie holds by construction	as planned
X5	Contract against NeMo Guardrails	Not run: the adapter was not built	not tested
H7	Stock Guardrails loop within 2 pp of the floor	Not met: 35.5 against 42.0 %, but not the 10 pp drop that rejects the prediction	as planned
H8	Typed schema without column list within 2 pp of the floor	Neither held nor rejected: the registered rule rejects only at 0/200, so the rule cannot register the large drop observed (4.0 against 42.0 %)	as planned
H9	Strict call-time enum ties the contract on entity	Held (2026-09-24); the run without the flag is `A-TIER-A-ONLY`; the flag went to the provider's standard endpoint and was not enforced	as planned
H10	Strict enum fails on an activated non-entity channel	Void with X2	not tested
H11	Our checks inside Guardrails match the contract on every channel	Held	as planned
H12	Wording effects differ between providers	Not run: the sweep ran on one provider	not tested
H13	Every contract arm reads 0 on every provider	Held, on the panel that ran	deviated
H14	The value channel activates under designed pressure	Not on leakage; the profile task activated the channel outside the registered design	not tested
H15	The omission channel activates under low anchor salience	Not run	not tested
H16	The post-strip anchor-coverage re-check fires on live data	Not exercised: nothing was stripped	not tested
Falsifier	Property-based falsifier with automated mutants	Built after the runs (10^6 cases); 36 hand-written mutation tests instead of generated mutants	deviated
Strict prompt	Replicate of the arms under the shipped prompt	Run for `A-L2` and `A-L3-SHIPPED` only, plus the unregistered `A-L2-RULES` and `A-L2-NOCLAIMS`	deviated
Panel	`deepseek-v4-flash`, `gpt-5.4-mini`, `Qwen2.5-3B` (vLLM)	`deepseek-chat`, `gpt-5.4-mini`, `qwen2.5:7b` (Ollama), without an amendment	deviated
Pooling	No pooled rates for floor, baseline, or sweep arms	None, except the share of invented names over the twelve A3 instances, which A3 registered pooled	as planned
Pilot	Report the June floor rates	15.0 and 11.5 % (rolling alias, records not kept; fault 5)	as planned
A3 R2a	On 12 instances frozen in July, misfiling dominates and invented names stay under 5 % of flagged responses	Held: 12 of 817 (1.5 %), on 1,888 valid floor responses	deviated
A3 R1, R2b	The verifier lowers delivered violations in a well-specified configuration (profile, $N = 600$; 12 instances)	Started, not completed: 93 and 91 of 600 profile responses, and 50 and 0 of 2,400 on the instances, when the provider balance ran out (A4); no test run, partial cells descriptive	not tested
A9.1	Naming interleaved on one harness: more wrong-number responses under suffixed names	7 against 1 of 200, Fisher $p = 0.068$: not rejected; all 7 suffixed wrong numbers carry the sibling's value	as planned
A11	The verifier lowers delivered violations in a well-specified configuration, arms interleaved (profile, natural names, $N = 600$ each)	10/600 against 0/600, Fisher $p = 0.002$; difference 1.7 points [0.7, 3.0]: supported	as planned
A14	Author-time enums covering 0, 3, 5, 8, 10 of the 10 evidence columns, interleaved: invented names fall as coverage rises (P1) and stay above the call-time list at 8 of 10 (P2)	143, 2, 0, 3, 0 of 200: P1 holds (trend $p < 10^{-76}$); P2 not supported (3 against 0, Fisher $p = 0.25$)	as planned
A13	One stronger LLM judge (`gpt-5.5`, pinned snapshot) passes the same validity check	Passed: 55/55 and 73/79 (64/65 after fault 17); misfiled `r2` read as a false relation in 40/40 (P1 fails; P2, P3 hold)	as planned

Table 3: The arms. Each fixes the system prompt, the tool schema and the check on the returned answer. *Open*: the answer's fields with no admissible values listed. *Call-time enum*: value lists built from E for each request. *Tier B*: our verifier with named retries. Leakage arms run on `deepseek-chat` with $N = 200$; the profile arms of RQ5 (prefix `P-`) mirror `A-L1`, `A-TIER-A-ONLY`, `A-CONTRACT`, `A-STRESS` and the two strong arms.

Arm	Prompt	Schema	Check
No check			
`A-L1` (floor)	bare	open	none
`A-L1-T0`	bare	open	none, $T = 0$
`A-L2`	shipped	open	none
`A-L2-NOCLAIMS`	shipped, no claim	open	none
`A-L2-RULES`	bare + three rules	open	none
`A-L1-DESCRIBED`*	bare	open + description	none
`A-STRICT-STATIC`	bare	typed, no column list	none
`A-STATIC-ENUM-STALE`	bare	typed + fixed list	none
`A-TIER-A-ONLY`	bare	call-time enum	none
`A-STRICT-ENUM`	bare	call-time enum + strict flag	none
Check and retry loop			
`A-GR-STOCK`	bare	open	Guardrails: structure
`A-GR-CHOICES`	bare	open	Guardrails: list from E
`A-GR-OURS`	bare	open	Guardrails + our checks
`A-CONTRACT`	bare	call-time enum	Tier B
`A-L3-SHIPPED`	shipped	call-time enum	Tier B
`A-STRESS`	bare	typed, no enum	Tier B
`A-STRESS-GENERIC`	bare	typed, no enum	Tier B, generic retry
Well-specified configuration, verifier varied			
`A-STRONG-NOVERIFY`	shipped	call-time enum + description	none
`A-STRONG-CONTRACT`	shipped	call-time enum + description	Tier B

*Also `A-L1-DESCRIBED-NEUTRAL`, whose description names no metric, and `A-L2-DESCRIBED` and `A-L2-DESCRIBED-NEUTRAL`, which use the shipped prompt.

enum is stale, and the profile arms of RQ5 were re-run under natural column names.

Sample size and dates Each arm on `deepseek-chat` and `gpt-5.4-mini` has $N = 200$, fixed in the analysis plan. The local `qwen2.5:7b` arms have $N = 20$, and the RQ4 variants have $N = 100$. One arm's responses on one task instance and date form a *cell*, and the arms run together on one date form a *battery*. A count of 0 in 200 bounds the rate below 1.9 % (Wilson, 95 %); Section 12 reports power. The provider serves a rolling model name, so the same arm can read differently on different days. Every rate therefore carries its run date, and arms are compared within a date.

For RQ4, the bare instruction was rewritten six ways (*terse*, *baseline*, *cautious*, *expert*, *helpful*, and *mechanical*), with $N = 100$ per variant. None of the variants contains a faithfulness rule, and all of them were written before the September runs. A June pilot measured the same sweep (fault 5, Section 10).

5.3 Measures

We score the three conditions on every response that reaches the user, once, into one table with a row per response, from which every count in this paper is computed (`scripts/unified_scoring.py`). A (C1) violation is a cited column or a claim's column outside $A_{E,\mathrm{col}}$. A (C2) violation is a claim on an admissible column whose pair E does not carry (a *pair not carried*) or whose number misses the measurement by more than τ (a *wrong number*). A (C3) violation, an *omission*, is a committed verdict that never references the anchor. A response *violates* the contract when the response has any of the three violations, and the tables give the channels separately.

Unconstrained arms name the statistic themselves. On the leakage task, the measurement's V_E therefore also accepts the usual names of the correlation (Pearson, Spearman, and their absolute forms) and the dataset-level quantities that E carries, whereas the verifier admits only the schema's statistic. Table 4 gives the rule, and one oracle scores every arm. A name that commits to a method resolves only to the method that E used. E keeps whichever of Pearson's and Spearman's coefficients is larger in magnitude, so "Pearson" on a column that E measured by Spearman is a pair that E does not carry. Of the 128,363 recorded claims that name a correlation on a listed column, 2,726 name E's method, and none names the other method. The verifier's own telemetry files a claim's inadmissible column under its value message; we score that case under (C1). The oracle's wider statistic set affects only the (C2) counts of unconstrained arms (the last column of the wording sweep, Section 8); an enforced arm's statistic field admits only `correlation`. Dataset-level quantities are accepted on any listed column, because unconstrained arms attach them that way and the number can be checked. The profile contract offers no slot for the task-level class balance, and Section 9 re-scores the profile task with one.

Table 4: How the leakage measurement resolves a claim's statistic name, after lower-casing and turning spaces and hyphens into underscores. The tolerance is $\tau = 0.005$ throughout; the verifier admits only `correlation`.

Statistic name	Resolves to
`correlation`, `corr`, `correlation_with_target`	the column's recorded correlation, signed
`pearson`, `spearman`, with `_correlation` or `_correlation_with_target`	the same, only if the name matches the method E recorded; otherwise a pair not carried
`abs_corr`, `max_abs_correlation`	its magnitude
`perfect_match_rate`, `row_count`	the dataset-level value, on any listed column
any other name	a pair not carried

Kinds of violation Every response that violates the contract also receives one *kind*, the first of the following four that applies. The kinds use only E and the frame's column list, so a program can assign them.

- *Extrinsic*: the response introduces something that E does not contain, namely an unlisted or invented name, or a number that appears nowhere in E.
- *Contradicts E*: for a pair that E carries, the response quotes a value that E records for a different item.
- *Misplaced*: the response uses only items that E contains, in a place that the contract does not admit, such as a misfiled name or a number that E carries quoted under a pair that E does not carry.

- *Omission*: the response commits to a verdict without referencing the anchor and breaks nothing else.

The kinds partition the violating responses, so the kinds sum to the contract count. A kind records where an item came from, not what a reader would understand. A misplaced item may be harmless to a reader, as a metric's name in the cited-column list may be. An item of the same kind can also state a false relation, as when a number that E records elsewhere is attached to a pair that E does not carry. That semantic judgment is the subject of the audit in Section 6. Table 5 sets out which question each measure answers.

Table 5: Four questions about a response, and what answers each. The first two are decided by program for every response; the third needs a reader, and one validated model judge answers part of the question; the fourth is outside the study.

Question	Answered by	Status
Does the response break the contract?	(C1)–(C3), by program	every response
Does each name or number appear in E or in the frame?	the kinds, by program	every response
Does the response state a false or unsupported relation?	human audit (A7) or model judges (A10, A12, A13)	A13 valid: misfiled names read as false; no human reading
Would the response mislead a reader?	—	not studied

A *response* is one narration as the narration reaches the user, however many provider calls its retries took; the replication package lists every run file with its responses and calls. We report percentages with Wilson 95 % confidence intervals (CIs) and the raw k/N. For an arm with a verifier, we report what the verifier did and do not read its (C1)–(C2) zero as a rate; its (C3) count is observed like any other. We exclude artifacts, namely a `null` column and several real names in one string. Two responses carry such an artifact, and counting the artifacts as violations changes one response in 19,132. Transport errors are excluded from N and reported. The registered runs of 2026-10-02 and 2026-10-03 (amendments A9, A11, and A14) are reported in their own paragraphs and are not part of the corpus counts; Table 6 lists every set of responses that the study recorded.

Table 7 collects the terms that the rest of the paper uses.

The tolerance decides almost nothing. Of the 100,642 claims on pairs that E carries, in the 10,941 responses of every arm without a verifier (both tasks, all providers), 99,008 reproduce the measured value exactly, and 1,608 round the value by less than 5×10^{-4}. Only a tolerance below that bound would begin to flag rounding. One claim drops a correlation's sign ($|\Delta| = 0.039$), and the remaining 25 miss by more than 0.05. The number of responses with a wrong number is 25 at $\tau = 5 \times 10^{-4}$ and 10^{-3}, 24 at 0.005 and 0.01, and 23 at 0.05 (`scripts/tau_sensitivity.py`). The narrators copy numbers, so the choice of τ moves few counts, but τ bounds what (C2) can see: (C2) checks closeness, not provenance, so a neighbour's value within τ of the pair's own value passes.

Table 6: What the study recorded. A response is one narration as delivered, after any retries; calls include the retries. The corpus feeds RQ1–RQ5; the registered runs are reported apart; the audit judges label the 204 items of the semantic audit (the inference models locally, the LLM judges through their providers).

Set	Responses	Calls	Section
Corpus, 2026-09-15 to 2026-09-30	19,132	19,968	RQ1–RQ5
`deepseek-chat`	17,522		
`gpt-5.4-mini`	1,450		
`qwen2.5:7b`	160		
A9.1 naming, interleaved	400	400	RQ5
A9.2 leakage retry audit	600	617	RQ2, RQ3
A11 verifier test, interleaved	1,200	1,211	RQ2, RQ3
A14 partly stale lists, interleaved	1,000	1,000	RQ3
Audit judges (A10, A12, A13)	1,020 labels	612	RQ1

Table 7: Terms used in the rest of the paper.

Term	Meaning
evidence E	the deterministic tool's output that the narrator explains
response	one narration as delivered to the user, after any retries
arm	a fixed prompt, schema, and check (Table 3)
cell, battery	one arm on one instance and date; the arms run together on one date
floor	the bare arm `A-L1`, with no enforcement
commits	has a verdict that is neither empty nor "cannot determine"
anchor	the top-ranked candidate, which a committed response must reference
misfiled	a name in E, placed in a column field that does not admit the name
unlisted	a real column of the frame that E does not list
invented	a name in neither E nor the frame
pair not carried	a claim whose (column, statistic) pair E does not record
wrong number	a claim whose number misses E's value by more than τ
omission	a committed response that never references the anchor
kind	extrinsic, contradicts E, misplaced, or omission

On the reference task the anchor (0.998993) and the distractor (0.998736) differ by 0.00026, and on the crowded instance adjacent columns lie 0.003 apart. Two crowded responses of `A-GR-CHOICES` carry nine values copied from the adjacent column that pass at $\tau = 0.005$; one of the two still violates through other values. A tolerance of 5×10^{-4} catches the crowded shifts, which is the one response that the count gains there, but not the reference pair. An absolute τ suits bounded quantities such as correlations, whereas unbounded quantities call for a relative tolerance and counts call for an exact match; each specification can set its own tolerance. Both of our specifications use 0.005, which on an integer count rejects every other integer. A fractional value within 0.005 of a count would pass, and none of the 24,298 recorded claims on integer quantities was such a value.

`deepseek-chat` is the model on which the failure occurs; `gpt-5.4-mini` and a local `qwen2.5:7b` are the null providers (Section 7.3). Temperature is 1.0, the default of every endpoint and the shipped narrator's setting, except in `A-L1-T0`. A contract arm with no violation at the default temperature is therefore not an artifact of a lowered temperature,

and the baselines' rates are conditioned on that temperature (at $T = 0$, the floor falls to 2.0 %; Section 6).

6 RQ1: How the Unconstrained Narrator Fails

On the bare arm, `deepseek-chat` cited a name outside the column domain in 42.0 % [35.4, 48.9] of 200 responses on 2026-09-15 (84/200). Four runs over nine days read 42.0, 43.0, 40.5, and 49.0 %, with no detectable heterogeneity (χ^2, $p = 0.34$). None of the 84 flagged responses of the first run contains an invented name.

Composition of the failures We examined every arm that runs the bare instruction with no column domain, in every run: sixteen cells of 200 responses on four task instances (`A-L1` in seven cells, `A-GR-STOCK` and `A-STRICT-STATIC` in four each, and `A-L1-T0` in one), with 1,012 flagged responses. Each of these responses places a real evidence name where the name is not admissible. The name is `r2`, which E carries as the name of the suspicious metric, written into the list of cited columns; three times, the name is `perfect_match_rate`, the name of another quantity in E. One response, on 2026-09-24, also writes `target`, a word that neither E nor the frame contains, because the frame calls its target column `y_true`. By our rule, that name is invented, although the word plausibly refers to the target column. In the terms of Section 2, the failure is intrinsic. The verifier's test is *outside the evidence*, which is weaker than *non-existent*: the verifier rejects a real but unlisted column just as the verifier rejects an invented name. The failure is not an artifact of synthetic data. On the two real datasets, the bare arm misfiles in 21.5 % [16.4, 27.7] (bodyfat) and 20.5 % [15.5, 26.6] (sambanis) of responses, every time with `r2`, and the shipped prompt and the contract read 0/200 on both datasets.

Twelve further instances On the twelve instances frozen in July, the floor arm cites an inadmissible name in 26.5 to 55.5 % of responses per instance, 817 of 1,888 responses in all. The 512 calls lost when the provider balance ran out (23 to 64 per instance, all with error 402 or 429) are excluded (amendment A4). These calls are each cell's last calls: the balance errors begin between the 140th and the 181st call, only 10 valid responses follow them, and the 52 rate-limit errors are spread through the runs. The retained responses are thus the earlier calls of each cell rather than a selection by content. Of the 817 flagged responses, 795 misfile `r2` or `perfect_match_rate`. Forty-five cite the frame's own target column, which E does not list, and 18 of these responses have no other violation. Twelve cite a name in neither E nor the frame: ten write `target` for the target column, and two respell a real column (`charged_est`, `total_charges`). The registered prediction that such names stay under 5 % of flagged responses holds (1.5 %). The 817 responses are thus 795 that misfile, 18 that cite only the target column, and 4 that cite only an invented name. Together with the 1,012 responses of the first four instances, all of which misfile, 1,807 of 1,829 flagged responses misfile, and 13 use an invented name. The composition found on four instances therefore holds on twelve instances that played no part in the design. The 1,829 responses are pooled from cells that differ in arm, instance, and date, so their composition describes the flagged responses, not an error rate over independent tasks.

An undescribed field The bare instruction says "list every column from the evidence you cite", and the open schema says nothing about the cited-column field. The suspicious metric is the subject of the investigation, which is a plausible reason why the model cites the metric. One sentence of field description takes the rate from 49.0 % (98/200, `A-L1` on 2026-09-24) to 0.5 % [0.1, 2.8] (1/200), whether the sentence names `r2` (`A-L1-DESCRIBED`, same day) or only says "not the names of metrics or statistics" (`A-L1-DESCRIBED-NEUTRAL`, next day). Rules do less on their own. The shipped prompt's first three rules, stated without enforcement claims, leave 7.0 % [4.2, 11.4] (14/200, `A-L2-RULES`), every one with `r2`. The full shipped prompt gives 0 % (0/200, below 1.9 % at 95 % confidence; `A-L2`), as the analysis plan predicted when the plan chose the bare prompt for the comparison (Table 2). Removing only the two sentences that claim violations are rejected leaves the shipped prompt at 0 % (0/200, `A-L2-NOCLAIMS`, 2026-09-27). The prompt's advantage over the three rules therefore comes from the rest of its content: four further rules, a verdict mapping, and its emphasis. Our reference names do not explain the misfiling: on the crowded instance, whose columns all have different names, the undescribed field gives 61.0 % [54.1, 67.5] (122/200), every one with `r2`. Nor does the failure occur on every model. `gpt-5.4-mini` never writes `r2` into a column field in 200 responses, and a local `qwen2.5:7b` never does so in 20. The description is therefore an effective intervention on this model. That the missing description is the only cause is more than these runs show, since temperature alone also lowers the rate and another model needs no description.

Temperature Rerun at $T = 0$ on 2026-09-27, the bare arm misfiles in 2.0 % [0.8, 5.0] of responses (4/200, `A-L1-T0`), every time with `r2`. The failure becomes rarer but does not disappear, and the temperature is one more setting chosen by whoever calls the endpoint.

What the violations introduce A response violates the contract when the response puts any item in a field that does not admit the item, whether or not the item is true. Table 8 therefore splits every violating response by the kinds defined in Section 5. A response that misfiles `r2` and also cites the target column counts as extrinsic, because the kind follows the first element that applies. On the bare leakage arms, 1,831 responses break the contract. Of these responses, 1,772 are misplaced only, 57 are extrinsic, 1 contradicts E, and 1 is an omission. Of the 57 extrinsic responses, 45 cite the frame's

own target column, which is true of the data but absent from E, and 13 use an invented name, 11 of them the word `target`; one response does both. The extrinsic kind is large in one place, an enum written in advance for other data, where 145 of 152 violating responses name a column that the frame does not have. On the profile task, the suffixed frame produces 19 responses that contradict E, and the natural-name frame produces 14 that use a name the frame lacks (Section 9). On this model, therefore, most contract violations introduce nothing that E lacks, and a contract rate would overstate invention many times over. The kinds do not settle whether a misplaced item misleads a reader. A misfiled `r2` misleads a program that consumes the field. Whether the same item misleads a person, and whether a number attached to the wrong pair states a false relation, are semantic questions.

A semantic reading Whether a misplaced item states a false relation is a semantic question. The registered blind audit (A7) was to answer the question on 204 items drawn from the flagged responses, each to be labeled A to E (Table 9). The authors decided not to label the sheet; labels from two of the authors would in any case not have been an independent validation. The sheet and its sampling script are in the replication package, with the key kept sealed, so that independent raters can still label the items blind. We registered three automatic substitutes in its place, each with the same validity check: on the items whose truth is fixed by construction, at least 90 % labeled correctly in each direction. Two natural language inference (NLI) models (A10) and two LLM judges (A12) failed the check (Appendix A.1). The third and last substitute (A13), `gpt-5.5` pinned to the snapshot of 2026-04-23, passed: the judge kept 55 of 55 items that are not false and caught 73 of 79 false or absent items. As registered, we report this judge's labels on the contested strata as one model's classification, validated on the items whose truth is fixed, and not as human judgment. The judges were tried in turn on the same items, so the judge that passed was not validated on items held out from the earlier attempts. The judge classified every misfiled `r2` (40 of 40, Wilson interval 91–100 %) as a false relation: a cited column of that name asserts a data column that the frame lacks. On numbers that E records under another pair, the classification is split. Of 44 such items, 13 are a correct value under the right column, filed under a statistic name that the contract's domain does not list (such as `pearson_corr_with_target`), which the contract counts as violations; eight move a value to another column; and 20 of the remaining 23 attach a dataset-level quantity that the sheet names without showing the number, so we leave those labels uninterpreted. Fault 17 (Section 10) moved 14 of the 44 items into this stratum and changed no judge's verdict.

Consequences On this model, the characteristic failure is the evidence's own vocabulary leaking into a field whose admissible set nobody stated. Such a response still violates the contract: a consumer that drops, correlates, or flags `r2` as a leak candidate acts on something that is not a column. The shipped panel prints the cited list under the heading "Evidence cited (verified against the evidence)", so an unverified `r2` would reach the user labeled as a verified evidence column. Every arm faced the same instruction, so the comparisons between arms stand.

Abstention A narrator that retreated into "cannot determine" would escape the (C3) obligation, and a narrator that also left its structured fields empty would score zero on every channel. Of the 46 abstentions on instances with a leak, 41 are on the crowded instance's stale-enum arm, whose list contains none of the evidence's columns, and `gpt-5.4-mini` never abstains. A further 13 responses carry an empty verdict

Table 8: Kinds of violation, with every response in exactly one row (`deepseek-chat` unless named). *Contract*: violates (C1), (C2), or (C3). The next four columns split the contract count by kind (Section 5) and sum to that count. *Extrinsic*: introduces a name or number that E lacks. *Contr.*: quotes, for a pair that E carries, a value that E records for another item. *Misplaced*: uses only items of E, in a place that the contract does not admit. *Omission*: only omits the anchor. *Invented*: responses with a name in neither E nor the frame, a subset of the extrinsic ones. The four-instance row exceeds the 1,012 entity flags of Section 6 by one omission and one wrong number. Generated by `scripts/unified_scoring.py`.

Responses	N	Contract	Extrinsic	Contr.	Misplaced	Omission	Invented
Leakage, bare instruction, four instances	3,200	1,014	1	1	1,011	1	1
Leakage, bare instruction, twelve instances	1,888	817	56	0	761	0	12
Leakage, six instruction wordings	600	370	21	1	348	0	20
Leakage, field described or shipped prompt	2,650	16	0	0	16	0	0
Leakage, enum bound to E, no verifier	600	0	0	0	0	0	0
Leakage, enum written in advance	800	152	145	0	0	7	145
Leakage, Guardrails with choices or our checks	1,400	3	0	2	1	0	0
Leakage, contract arms	4,000	0	0	0	0	0	0
Profile, suffixed names, no verifier	600	29	2	19	8	0	1
Profile, natural names, no verifier	693	36	14	0	22	0	14
Profile, contract arms	1,091	0	0	0	0	0	0
`gpt-5.4-mini`, every arm	1,450	3	0	0	3	0	0
`qwen2.5:7b`, every arm	160	2	2	0	0	0	2

Table 9: The semantic audit. Top: the labels of the judge that passed its validity check (`gpt-5.5`, amendment A13) per stratum, with the strata corrected for fault 17. A: supported, in place; B: right information, wrong place; C: a false or unsupported relation; D: a name or number not in the evidence; E: cannot tell. The truth is fixed for S3–S9 and contested for S1 and S2. Bottom: each judge's validity check, as items that are not false kept as supported (A or B) and false or absent items caught (C or D); the bar is 90 % each way. Generated by `scripts/audit_table.py`.

Stratum	N	A	B	C	D	E
S1 misfiled name	40	0	0	40	0	0
S2 number under another pair	44	13	0	8	23	0
S3 another item's number	23	1	0	21	1	0
S4 real column E lacks	15	15	0	0	0	0
S5 invented, stale list	10	0	0	0	10	0
S6 invented name	30	0	0	0	30	0
S7 number in no part of E	2	0	0	1	1	0
S8 admissible name	20	20	0	0	0	0
S9 claim that matches E	20	20	0	0	0	0

Judge	kept	caught	
DeBERTa (NLI)	55/55	48/65	invalid
RoBERTa (NLI)	53/55	21/65	invalid
`gpt-5.4-mini`	41/55	54/65	invalid
`deepseek-chat`	48/55	63/65	invalid
`gpt-5.5`	55/55	64/65	**valid**

(8 of 160 on `qwen2.5:7b` and 5 on `deepseek-chat`), all on arms without a verifier. The zeros of the enforced arms therefore come from narrators that committed to a verdict.

Answer to RQ1 The failing model writes the evidence's own vocabulary, above all `r2`, into an undescribed column field. Of the 1,829 flagged responses, 1,807 misfile and 13 use an invented name, and most contract violations introduce nothing that E lacks. The one model judge that passed its validity check classified every audited `r2` in a column field as a false relation. On the reference task, a field description, the shipped prompt, or temperature 0 lowers the rate from 42–49 % to 0–2 %.

7 RQ2 and RQ3: Enforcement, Its Cost, and the Alternatives

For the arms with a verifier, a delivered count of zero on (C1) and (C2) is a property of the design and no finding in itself: whatever still violates the contract after the retries is deleted (Section 3). The empirical questions are three. Does the implementation realize the property? What does enforcement cost in retries, latency, and delivered content? And does the verifier add anything to a configuration that is already well specified? This section answers the second and the third; the mutation, fault-injection, and falsifier tests of Section 3 answer the first.

Table 10: Percentage of responses reaching the user with a structured violation, 200 per cell, `deepseek-chat`. *Date* is the 2026 run date of the reference cell; the crowded instance, whose evidence columns the author-time enum does not cover, ran on 2026-09-24/25, and the strong arms ran on 2026-09-30 on both. A sampled 0 bounds the rate below 1.9 % (Wilson, 95 %). For the arms with a verifier (below the second rule) the zero on (C1) and (C2) holds by construction and the zero on (C3) is observed; Table 11 reports what the verifier did. Table 3 defines the arms.

Arm	Mechanism	Date	Reference (%)	Crowded (%)
`A-L1`	no enforcement	09-15	42.0	61.0
`A-L1-T0`	no enforcement, $T = 0$	09-27	2.0	—
`A-GR-STOCK`	Guardrails, no faithfulness check	09-15	35.5	57.5
`A-STRICT-STATIC`	schema, no column enum	09-15	4.0	27.5
`A-STATIC-ENUM-STALE`	author-time column enum	09-15	0[a]	76.0[b]
`A-L2`	shipped prompt	09-24	0	—
`A-L2-NOCLAIMS`	shipped, no enforcement claim	09-27	0	—
`A-L2-RULES`	its first three rules only	09-25	7.0	—
`A-L1-DESCRIBED`	description naming `r2`	09-24	0.5	—
`A-L1-DESCRIBED-NEUTRAL`	description, no metric named	09-25	0.5	—
`A-TIER-A-ONLY`	call-time enum	09-16	0	0
`A-STRICT-ENUM`	call-time enum, strict flag	09-24	0	—
`A-GR-CHOICES`	Guardrails, choices bound to E	09-18[c]	0	1.5[d]
`A-GR-OURS`	Guardrails, our checks	09-15	0	0
`A-CONTRACT`	Tier A + Tier B	09-15	0	0
`A-STRESS`	Tier B alone	09-15	0	0
`A-L3-SHIPPED`	as shipped, prompt included	09-24	0	—
`A-STRONG-NOVERIFY`	shipped + enum + description	09-30	0	0
`A-STRONG-CONTRACT`	the same + Tier B	09-30	0	0

[a] On the reference task the author-time list equals the evidence's ten columns, so the arm is identical to the call-time enum. [b] 152 of 200: 145 cite invented names (72.5 %), 132 of them also omitting the anchor, which the list does not contain, and 7 more commit while citing nothing; 41 abstain. [c] `A-GR-STOCK` read 43.5 that day. [d] 3 of 200, with one misfiled name in a claim's column field and two misquoted values, in the fields a single choice list does not check.

7.1 Comparing the mechanisms

Table 10 gives the rates. On the reference task, every arm that tells the model the admissible column set reaches zero or one violation in 200; on the profiling task, the enumeration let column names through (Section 9). The arm may state the set in the shipped prompt, in a description on the field, in an enum in the schema, in a choice list inside a guardrail loop, or through our verifier. The one exception is the shipped prompt's first three rules on their own, at 7.0 %. A typed schema without a column enum reads 4.0 %, and the arms that state nothing read 35–49 % across runs.

The cleanest comparison is the same-date one inside a single toolkit. On 2026-09-18, `A-GR-STOCK` and `A-GR-CHOICES` ran the same Guard, reask loop, budget, prompt, and model, and read 43.5 % (87/200) against 0 % (0/200, with 88 responses sent back and repaired). The stock configuration validates structure and has no faithfulness validator, because a fresh install registers none, so nothing in that configuration could reject `r2`. The choice arm is our reimplementation of the toolkit's acceptable-values semantics, with its list built from E at call time, and the choice arm checks the cited-column list and nothing else. On the reference task, the choice arm matched our validator (`A-GR-OURS`, where no violation reached the user and 84 responses were repaired). On the crowded instance, the choice arm let three responses through, one with a misfiled name in a claim's column field and two with misquoted values, whereas our validator in the same loop let none through (118 repairs). The binding does the work on the field that the binding covers, and Tier B's other checks cover the fields that a choice list leaves open.

`A-STRICT-STATIC`, a schema with types and required fields but no column enum, reads 4.0–7.0 % across runs, against the floor's 42 %. A schema therefore steers a model strongly even where the schema constrains nothing. This result contradicts the prediction of H8 (a rate within 2 points of the floor) without meeting the registered rejection rule, which required 0/200 (Table 2). The evidence-bound enum without verification left no violation on the reference task with or without the provider's strict flag, so hypothesis H9 holds on the entity channel. Neither result amounts to a guarantee. In the non-strict arms, the enum ran as a request, and whether a strict flag is honored is up to the provider. None of the 400 responses sent with the strict flag carried a column outside the enum, but from outside the provider, a model that obeyed and a server that quietly replaced a value look the same.

Against a well-specified configuration Each alternative above lacks something. By *well specified*, we mean the best configuration that this study found, not the strongest one possible: a relational schema that encodes (C2) was not tried. The two strong arms give the narrator the shipped prompt, the call-time enum, and the neutral field description, and the arms differ only in whether the payload is verified. On 2026-09-30, both arms read 0/200 on the reference instance, the crowded instance, and the two real datasets. On the twelve frozen instances, the budget left 50 unverified responses, none with a

violation, and no verified ones (A4). On leakage, the verifier never had to send a response back, and the median latencies of the two arms were within 0.1 s of each other (2.4–2.7 s). In a configuration this well specified, we observed no gain in rate on leakage. Pooled over the four instances (800 responses per arm, an exploratory pooling), the difference is 0, with a Newcombe interval of −0.5 to 0.5 points. A benefit larger than half a point on these instances is therefore unlikely; the interval lies inside the plan's 2-point equivalence band, although this comparison was not registered.

On the profile task, an exploratory run under natural names gave the one difference that we observed: 4 of 200 unverified responses against none verified (Fisher $p = 0.12$; difference 2.0 points, Newcombe interval −0.2 to 5.0). The unverified violations cite `reference_count` beside the real `referral_count`, attach real numbers to invented `*_placeholder` columns, or attach a number to a statistic that E lacks. The confirmatory test that we registered next (amendment A3) had a profile leg (R1) and a leg on the twelve frozen instances (R2b). Both stopped when the provider balance ran out, R1 after 93 and 91 responses and R2b after 50 unverified responses, none with a violation, and we run no test on the partial cells. What the verifier adds is a check whose result does not depend on the configuration.

The registered test A3 had registered that its run would not be repeated or extended. When budget returned, we registered a new test on the profile task alone, before any call (amendment A11), which is not pooled with R1. On the profile task under natural names, the two strong arms ran 600 responses each, one call of each arm per index in an order drawn at random per index, on one harness commit at temperature 1.0, with every attempt kept. One interim look at the partial unverified records (5 violating of 358) is recorded in the plan; no decision depended on the look, and the run was neither stopped nor extended. The unverified arm delivered a violation in 10 of 600 responses (1.7 % [0.9, 3.0]) and the verified arm in none (0 % [0, 0.6]). The registered two-sided Fisher test rejects ($p = 0.002$), and the difference of 1.7 points has a Newcombe 95 % interval of 0.7 to 3.0. The point difference lies below the plan's 5-point materiality bar, and so does the whole interval, so the difference is significant but not material; A11 registered its own consequence, to report a benefit of the observed size. The ten violations are slips rather than the misfiling of the bare instruction, and nine of them fall outside the enumerations that Tier A writes, so a provider that enforced the schema would also have excluded them. None is a wrong number for a carried pair, so A11 does not exercise the tolerance of the value channel, and the case for the check over an enforced schema rests on the values and the anchor. Two cite a statistic name (`cardinality`, `zero_ratio`) as a column. Four claim a pair that E does not carry, three of them under a statistic name that the profiler does not use, such as the misspelled `missding_count`. Four name a column that the frame lacks: two misspell a real column (`season_minutes`, `tenanture_months`), one cites two placeholder columns, and one invents a `product` column. The verified arm sent back ten responses, and the independent reading of the rejection audit below confirms all eleven rejected attempts. One of those attempts narrated the columns of a different, well-known dataset (`crim`, `zn`, …, `medv`). The verifier cost 11 extra calls in 600 (1.8 %), and we observed no reduction in the content measures: 29.4 correct claims per response in each arm and the anchor's value in every response, at median latencies of 5.47 and 5.44 s. These measures do not establish that the answers are equivalent. In the best configuration that we found, the verifier therefore removes a small but measurable residue of violations with no observed loss in what we measured.

An author-time enum on new data On the reference task, the author-time list is the evidence's own column set, which makes the stale-enum arm the call-time arm under another name. The two arms diverge on the crowded instance. There, the list written against the reference task covers none of the ten evidence columns, and the model obeys the list. The model cites reference-task columns that do not exist in this frame in 145 of 200 responses (72.5 % [65.9, 78.2]). Every one of these citations is an invention, a failure that the undescribed field never produced. In addition, 139 responses omit the anchor, which the list does not allow them to name (132 of the 145, and 7 that commit while citing nothing), and 41 responses abstain. The same instance reads 27.5 % (55/200) with no column enum and 0 % (0/200) with the enum built at call time. On evidence that the list predates, the author-time list produces the failure that the list was supposed to prevent.

A registered, interleaved test varied how stale the list is (amendment A14). Lists of ten entries covered 0, 3, 5, 8, or 10 of the crowded instance's ten evidence columns, the rest taken from the reference list; every list that covered any column included the anchor, and the lists were nested. Responses with an invented name fell from 143 of 200 without coverage, which reproduces the earlier stale arm, to 2, 0, 3, and 0 of 200 (Cochran–Armitage test for trend, $p < 10^{-76}$). At 8 of 10 columns the residual did not differ from the list built at call time (3 against 0 of 200, Fisher $p = 0.25$), so the second registered prediction is not supported. The remaining inventions cite stale entries that the list still carries, such as `feature_12`. The failure of the author-time list is thus concentrated in a list that misses the evidence and the anchor; a partly stale list that keeps the anchor behaves, at this resolution, like the list built at call time. Because every covering list included the anchor, the test does not separate coverage of the anchor from coverage of the other columns.

7.2 Cost of verification

Table 11 shows what an enforced arm's zero consists of. One named retry repaired every violation that the verifier met on these arms; only the content-free correction of `A-STRESS-GENERIC` needed a second retry, twice in 200. In the registered test A11, one response needed a second retry, because the first retry cited another column that the frame lacks. Nothing was stripped anywhere in the study, so the deletion that

Table 11: What the verifier did on the enforced arms (reference battery and the profile task), $N = 200$ each. A response is *clean* when its first attempt satisfied (C1)–(C2); otherwise one named retry repaired the response. No response on these arms needed a second retry, and none was stripped. `A-GR-CHOICES` ran on 2026-09-18. Calls and wall-clock latency are per response; `A-L1` and `P-L1` are the unenforced baselines.

Arm	clean	retried	calls	latency
`A-L1`	—	—	1.00	2.8 s
`A-CONTRACT`	200	0	1.00	3.3 s
`A-STRESS`	188	12	1.06	3.6 s
`A-GR-OURS`	116	84	1.42	4.5 s
`A-GR-CHOICES`	112	88	1.44	4.9 s
`P-L1`	—	—	1.00	5.0 s
`P-CONTRACT`	194	6	1.03	5.5 s
`P-STRESS`	187	13	1.07	5.0 s

guarantees (C1)–(C2) remained a worst case that never occurred.

Three costs are distinct: the local check, the larger request schema, and the extra calls of a retry. With nothing to repair, the contract arm took 3.3 s per response against 2.8 s for the floor (medians 3.3 and 2.8 s; 95th percentiles 3.9 and 3.3 s). The check is not what costs time. Offline, verification and stripping take a median of 17 μs per response (95th percentile 41 μs), from 7 μs for responses with fewer than five cited names and claims to 18 μs for responses with twenty or more. The timings cover all 19,132 recorded responses on one machine, each the median of five repetitions (`scripts/time_verifier.py`). Binding E takes 15 μs per call. `A-TIER-A-ONLY`, which sends the same enum schema without verifying, took 3.5 s on another date. The difference is therefore not the check. The larger schema is the likely cause, although runs on different dates cannot separate the schema from provider variation.

Arms whose responses need retries cost more on average: 3.6 s per response on `A-STRESS` and 4.5–4.9 s on the Guardrails arms. Removing the enum while keeping the loop and the verifier raises the number of retried responses from 0 to 12 on leakage and from 6 to 13 on the profile task. The schema does not strengthen the guarantee; the schema reduces retries on the channel that the schema can express. Retry counts vary between runs (Section 12), so the cost has to be reported per run, whereas the guarantee does not depend on the cost. Appendix A.5 lists results that appear in neither table.

Table 12 shows what reached the user. All responses on the reference task but one, on `A-GR-STOCK`, delivered at least one claim; none abstained, and none lacked an admissible verdict. No zero is therefore the vacuous zero that an empty answer would give. Nearly every delivered claim matches E, and every arm reports the anchor's measured correlation in at least 99.5 % of its responses. The arms that carry the shipped prompt report little beyond the anchor: they deliver about a quarter of the correct claims that the bare and contract arms deliver (2.2–3.1 against 9.6–9.8 per response). Part of what the shipped prompt buys is therefore a narrower answer, whereas the contract keeps the content of the bare answer.

Table 12: What reached the user on the reference task, `deepseek-chat`, $N = 200$ per arm: responses with any violation, claims delivered per response, claims per response that match E within τ, the share of responses that report the anchor's measured correlation, responses delivering at least one claim, abstentions, responses with no admissible verdict, and responses the arm's loop sent back. Generated by `scripts/delivery_table.py`.

Arm	any, %	claims	correct	anchor, %	≥1 claim	abstain	no verdict	retried
`A-L1`	42.0	9.6	9.6	100.0	200	0	0	0
`A-L2`	0.0	2.2	2.2	100.0	200	0	0	0
`A-GR-STOCK`	35.5	9.8	9.8	99.5	199	0	0	0
`A-STRICT-STATIC`	4.0	7.1	7.1	100.0	200	0	0	0
`A-GR-OURS`	0.0	9.8	9.8	100.0	200	0	0	84
`A-CONTRACT`	0.0	9.7	9.7	100.0	200	0	0	0
`A-STRESS`	0.0	9.7	9.7	100.0	200	0	0	12
`A-L3-SHIPPED`	0.0	2.5	2.5	100.0	200	0	0	1
`A-STRONG-NOVERIFY`	0.0	3.1	3.1	100.0	200	0	0	0
`A-STRONG-CONTRACT`	0.0	2.9	2.9	100.0	200	0	0	0

Auditing the rejections A retry's success can be checked only if the rejected attempt is kept. The leakage records keep the delivered payload and the kind of each rejection, but not the rejected payloads, so a leakage rejection can be re-judged only through the verifier's own telemetry. Profile records written from 2026-09-24 on keep every attempt. Table A2 in Appendix A lists all 24 rejected attempts that these records hold, 11 of them from the registered test A11, each re-judged by a script that reads only the evidence dump and the stored attempt, and no contract code (`scripts/rejudge_rejections.py`). The independent reading confirms every rejection. Seven of the twelve content rejections are wrong numbers, and each of the seven is the measurement of the categorical column that shares the numeric column's suffix, such as `cat_feature_07`'s missing rate quoted for `num_feature_07`. Four rejections name a column that E does not list, and one claims a pair that E does not record. The thirteenth attempt had no admissible verdict. Of the eleven rejections in A11, ten name a column that the frame lacks and one claims a pair that E does not record. Every retry dropped or corrected the offending item, and every final attempt is clean. On balance, the retry does not shed acceptable content. Across the twelve content rejections of the earlier runs, the rejected attempts carried 260 claims that match E and the final attempts 258, although the narrator writes a new answer rather than editing the old one: 96 correct claims were dropped and 94 added (`scripts/retry_content.py`). In A11, the ten retried responses carried 236 correct claims on their first attempts and 278 on their final ones (89 dropped, 131 added). On leakage, responses that went through a retry deliver about as many correct claims as responses accepted at once, for example 9.54 against 10.00 on `A-GR-OURS` and 9.58 against 9.65 on `A-STRESS`. The leakage harness has kept every attempt since 2026-10-01, and a registered re-run on the crowded instance used that version (amendment A9.2, 2026-10-02). There, the floor violated in 114 of 200 responses (57.0 %), `A-CONTRACT` and `A-STRESS` delivered none, and `A-STRESS` sent back 17 responses. The second scorer (Section 10), which shares no

code with the verifier, confirms all 17 rejected attempts, each for a name outside E, and finds all 17 final attempts clean. The retried responses carried 170 correct claims before the retry and 170 after.

Why Tier B is still needed Our schemas bind each field to an independent `enum`. That form expresses the column domain, which is why the enum arms tie the contract on leakage, but the form does not express the other two channels. A value's admissible set is a tolerance band that depends on the column and statistic that a sibling field chose, and "a committed verdict addresses the anchor" is a property of the whole response. Full JSON Schema could express both conditions for a finite E, with one `oneOf` branch per (column, statistic) pair carrying `minimum` and `maximum`, and with `if/then` and `contains` for the anchor. Provider strict modes accept subsets of JSON Schema, such a schema grows with $|V_E|$, and we did not test one. Whatever the schema, the schema's enforcement happens inside the provider, whereas Tier B checks the returned payload in our own process. Where logits are available, grammar-constrained decoding could enforce (C1) by construction and, with a grammar generated for each call, encode more of (C2) and (C3). We did not run that comparison: the one open model that we ran, `qwen2.5:7b`, did not fail without enforcement, so there was nothing for constrained decoding to prevent.

7.3 Providers without failures

On `gpt-5.4-mini` (same evidence, prompts, scorer, and budget, and no transport errors), all seven arms that existed at the time show no violation in 200, the floor included. That model cites exactly the ten evidence columns in 199 of 200 responses and never writes `r2` into a column field. A local `qwen2.5:7b` ($N = 20$, so a 0 bounds its rate only below 16.1 %) shows no violation except on `A-STRICT-STATIC`, at 10 % (2/20), both times with the invented name `suspicious_metric.name`. These providers offer nothing to compare, so the comparative claims of this section rest on one model, and its rates describe `deepseek-chat` on particular dates, not LLMs in general.

Answer to RQ2 and RQ3 In every configuration that we ran, no (C1) or (C2) violation reached the user under the contract, and named retries repaired each violation, at the first attempt in all but three responses. The local check costs microseconds, and retries add calls. Every rejection that the records keep, 24 on the profile task and 17 in a registered leakage re-run, was confirmed by a reading that shares no code with the verifier. In the best configuration that we found, the registered interleaved test measured a benefit of the verifier: 10 against 0 violating responses in 600 per arm (Fisher $p = 0.002$), with no observed reduction in the content measures. In these configurations, a stock guardrail loop stays near the floor, and an enum written for other data causes invention. An enum or choice list bound to E matches the contract only on the column field, and the toolkit running our checks matches the contract. The comparisons measure what each configuration checks.

8 RQ4: Wording and Specification Move Compliance

Wording and specification Table 13 gives the six rule-free variants of the bare instruction. Over identical evidence and schema, the variants move the blended entity rate from 9.0 % to 72.0 %. Wording is already known to move behavior this much (Sclar et al., 2024). The variants also differ in how much of the answer they specify, and the count of claims on pairs that E does not carry follows that ordering across the six variants. The count of responses with such a claim is 0 and 1 in 100 where the instruction names the statistic, 8 and 10 where the instruction mentions correlations, 26 where the instruction names the field without saying what goes in the field, and 45 where the instruction says nothing. On the entity channel, the baseline instruction's "list every column from the evidence you cite" produces pure misfiling, and Section 6 shows that describing the field removes the misfiling.

Out-of-evidence names Of the 20 names in 600 responses that E does not carry, 18 are the word `target`. The other two, both from *expert*, name a derived quantity and a group of columns. By our rule, all 20 names are invented, because neither E nor the frame contains them, although the word plausibly refers to the target column. A carried number is misquoted once. Across variants, the out-of-evidence and misfiled counts show no detectable association (Spearman $\rho = -0.25$, $p = 0.63$), although with six variants the test has almost no power. *Terse* has the lowest blended rate and the most out-of-evidence names, but 7 against *mechanical*'s 1 is not significant ($p = 0.065$).

Interpretation Measured compliance belongs to a configuration: the wording, what the schema says about each field, the names in the data, the provider, and the date. Two diagnostic arms show the same from inside the contract. The evidence-bound enum alone, with no verification, left no violation in 200 responses (`A-TIER-A-ONLY`), and the full contract under the variant that fails most often without enforcement produced no violation on any first attempt.

Answer to RQ4 The wording, and how much the wording specifies, moves the rate from 9 to 72 %, and the count of claims on pairs that E does not carry follows how much of the answer the instruction specifies. The effect of a wording is known only for the configuration in which the wording was measured.

9 RQ5: Does the Contract Transpose?

The second narrated artifact is a deterministic dataset profiler that ships in the same tool. The profiler reports, per column,

Table 13: Rule-free instruction variants, $N = 100$ each, `deepseek-chat`, run on 2026-09-15, ordered by the blended entity rate, the share of responses with any entity violation. *Outside E* is a name E does not carry (18 of the 20 are the word `target`, which neither E nor the frame contains; the frame's target column is `y_true`); *misf.* is a name in E that is not a column; *misq.* is a wrong number on a pair E carries; *unver.* is a pair E does not carry. *Statistic specified* is what the instruction says about the claim's statistic.

		entity channel				value	
Variant	Statistic specified	blended	95 % CI	outside E	misf.	misq.	unver.
terse	`"correlation"` named	9.0 %	[4.8, 16.2]	7	4	0	1
baseline	`"correlation"` named	36.0 %	[27.3, 45.8]	0	36	0	0
expert	nothing	56.0 %	[46.2, 65.3]	5	51	0	45
cautious	"each correlation"	68.0 %	[58.3, 76.3]	2	68	1	8
helpful	"every correlation number"	71.0 %	[61.5, 79.0]	5	71	0	10
mechanical	the field, not its content	72.0 %	[62.5, 79.9]	1	71	0	26

the type, missing counts and rates, descriptive statistics, cardinality, and outlier counts, plus a task-level class balance. We bind the profiler through the contract layer of Section 4.2, with no branch that names either task. The shape differs from leakage: the entity domain is every column of the frame (24 columns, against 10), the statistic domain has 13 names against 1 and depends on the column, and E carries 196 measured quantities against 10. The evidence comes from running the profiler on a frame with ordinary data-quality problems: mixed types, missingness that rises across the categorical block, and injected outliers. That frame names its columns `num_feature_`k and `cat_feature_`k, so a numeric and a categorical column share each suffix. We also ran the unverified and strong arms on the same data under distinct natural names (`age`, `region`, …) that share no suffix.

Table 14: The second task, $N = 200$ per arm, `deepseek-chat`, counted in responses; suffixed-name arms ran on 2026-09-19, natural-name arms on 2026-09-24, the strong arms on 2026-09-30. *C1*: a name outside the column domain, in the cited list or a claim's column. *Wrong no.*: a claim on a pair E carries whose value misses the measurement. *Not carried*: a claim on an admissible column whose pair E does not carry; every one quotes a number E carries elsewhere. *Tier B*: responses the verifier sent back, for content / for a verdict outside its enum. Generated by `scripts/unified_scoring.py`.

Arm	Names	C1	wrong no.	not carried	any, % [95 % CI]	Tier B
`P-L1`	suffixed	0	9	9	8.5 [5.4, 13.2]	—
`P-TIER-A-ONLY`	suffixed	1	11	0	6.0 [3.5, 10.2]	—
`P-CONTRACT`	suffixed	0	0	0	0	6/0
`P-STRESS`	suffixed	0	0	0	0	13/0
`P-L1`	natural	8	0	20	13.0 [9.0, 18.4]	—
`P-TIER-A-ONLY`	natural	5	0	1	3.0 [1.4, 6.4]	—
`P-STRONG-NOVERIFY`	natural	3	0	1	2.0 [0.8, 5.0]	—
`P-STRONG-CONTRACT`	natural	0	0	0	0	3/1
`P-STRONG-NOVERIFY`	suffixed	0	0	0	0	—
`P-STRONG-CONTRACT`	suffixed	0	0	0	0	0/0

Wrong numbers and column naming Table 14 gives the counts. On the suffixed frame, the unconstrained arm put a wrong number in 9 of 200 responses (4.5 % [2.4, 8.3]). Eleven of the arm's 12 wrong numbers are a real measurement of the column that shares the suffix, such as `cat_feature_07`'s missing rate reported for `num_feature_07`, and one appears nowhere in E. Under distinct names, in the runs of 2026-09-24, the same model on the same data put a wrong number in none of 400 unverified responses. The match with the column that shares the suffix is consistent with confusion between the names that our generator chose. A match, however, does not show where the model took the number from. The comparison across naming schemes is weaker evidence. The main suffixed runs (2026-09-19) and the natural-name runs (2026-09-24) are five days apart, in separate blocks, with harness and adapter changes between them and a model name that may point to another version (Section 12).

The records allow one cleaner comparison. On 2026-09-24, a re-run of `P-CONTRACT` on the suffixed frame kept every attempt, and that arm's first attempt is generated exactly as a `P-TIER-A-ONLY` response is, with the same prompt, enum schema, and temperature. Seven of the 200 first attempts quoted a categorical sibling's missing rate or count for a numeric column (Table A2). The natural-name `P-TIER-A-ONLY` arm, run about eight and a half hours later on the same day, put a wrong number in none of 200 responses (Fisher $p = 0.015$). Between the two runs, the generation code did not change: the only changes were the naming option and a verifier fix that acts after generation. That comparison was not registered, and the two runs were not interleaved, so the comparison is exploratory.

The registered interleaved run To separate the naming from the date and the harness, we registered a run that calls both naming schemes on one harness commit, one call of each per index, in an order drawn at random per index (amendment A9.1). On 2026-10-02, with the enum arm and the model served that day, 7 of 200 suffixed responses and 1 of 200 natural-name responses carried a wrong number (two-sided Fisher $p = 0.068$). The registered test therefore does not reject at the 0.05 level, and, as registered, we no longer count the same-day comparison as support for the naming explanation. Descriptively, all seven suffixed wrong numbers equal the value that the same-suffix sibling records for the same statistic, and the one natural-name wrong number does not. The naming explanation therefore remains exploratory: which

column each wrong number matches is consistent with name confusion, and the controlled comparison of rates points the same way without reaching significance.

Correct quantities in inadmissible slots Every claim on a pair that E does not carry, on both frames, quotes a number that E really carries, and quotes that number correctly. Most of these claims are the class balance, which `gpt-5.4-mini` also misplaces in 3 of 50 probe responses. The profiler reports the class balance at task level, the narrator attaches the figure to the target column, and the contract admits only per-column statistics, so the correct figure has no admissible slot and the verifier rejects the claim (fault 13 had counted such claims as invented). A program that consumes the claim slot receives a wrongly keyed value. Excluding the class balance is a design choice of ours. Re-scored with the class shares admitted on the target column, the unconstrained arm falls from 26 to 10 violating responses under natural names and from 17 to 10 under suffixed names, and the three `gpt-5.4-mini` violations disappear.

What remains under suffixed names is the wrong numbers. Under natural names, two things remain. Some responses key real quantities, such as the frame's shape, to a column or to a pseudo-column such as `shape`. Five responses name a column that the frame does not have, two of them respellings of real columns (`logging_count`, `acquired_channel`). Distinct names thus removed the wrong numbers and brought a few invented columns. Across the 693 unverified natural-name responses, 11 name a column that the frame does not have, leaving aside pseudo-columns for the frame's shape, against 1 of 600 under suffixed names.

How the tiers separate Binding the enum removes pairs that E does not carry almost entirely (20 to 1 under natural names), lowers inadmissible column names less (8 to 5), and leaves wrong numbers in place (9 against 11 under suffixed names). An enum can refuse a pair outside the admissible statistics, but an enum cannot constrain a number whose admissible set is a band around a value chosen by a sibling field. Tier B handles both cases. `P-CONTRACT` returned no violation and sent back 6 responses, and `P-STRESS`, with the enum removed, sent back 13. The re-run of `P-CONTRACT` that kept every attempt (200 responses, 8 catches) found no entity violation on any first attempt. The enum therefore held on 399 of 400 first attempts across the two runs, and we have no explanation for the one exception. The enum did not hold everywhere: without the strict flag, 5 of 200 natural-name `P-TIER-A-ONLY` responses put a name outside the column domain into a column field. The re-run also caught an interquartile-range outlier count claimed for `cat_feature_03`: the column and the statistic are each admissible, but E does not carry the pair. A per-field enum cannot express that relation, and the domain check of (C2) does.

Scope of the result The contract binds to a second evidence shape without task-specific verification, and the two tiers separate once the failing channel is one that the per-field enums we use cannot express. The result does not show that misfiling numbers is a general property of narrators. On this model, the wrong numbers are consistent with confusion between our names, and what survived distinct names reflects our binding choices as much as model error.

Answer to RQ5 The contract binds to a second evidence shape without task-specific code. There, Tier B catches wrong numbers that the per-field enums we use cannot express. Eleven of the 12 wrong numbers of the first run, and all seven suffixed ones of the registered run, match the measurement of the column that shares the suffix. That pattern is consistent with confusion between our generated names, but the registered interleaved run, 7 against 1 wrong-number responses in 200 ($p = 0.068$), did not reach significance, so the explanation remains exploratory.

10 Corrections to Our Own Instruments

Every number in this paper is recomputed by a script from immutable records of the live responses, so correcting an instrument changes a derived measurement, never an experimental record, and the replication package keeps each superseded measurement beside its correction. Two scorers written independently from the definitions, sharing no code, agree on every entity flag, misfiled flag, and out-of-evidence flag over all 16,698 leakage records. Table 15 lists the seventeen faults that we found in our instruments, how we found each fault, and what each fault changed. Faults 1, 3, 6, and 7 concern the downstream benchmark (Appendix A.6); Appendix A.2 describes faults 14 to 17 and the few flags on which the two scorers differ.

Three patterns recur. In faults 2, 12, and 13, a scorer reported the narrator wrong for being right; fault 9 was a control that silently received the treatment; and the first corrections to the downstream checklist all moved toward our hypothesis. The corrections that broke that pattern came from procedures designed to look for such faults (Appendix A.2).

We also found defects in the artifact itself, which the table does not count because the artifact is not an instrument. A claim value of NaN passed the verifier, because every comparison with NaN is false. The property-based falsifier that the analysis plan registered, which we built late, reproduces that refutation at its third generated case on the earlier code. The falsifier then found a second defect: a 400-digit integer, which a JSON parser returns as an exact integer, crashed the value check. Both defects are fixed, and the fixed verifier passes 10^6 generated cases. The NaN fix came on 2026-09-24, after the main battery; no recorded response carries either value, so no reported number changes.

Table 15: Instrument faults, numbered in the order found.

#	Fault	Found by	Moved
1	checklist penalized unlisted correct forms	authors	defects 40/31/15 → 31/24/8
2	value scorer ignored the statistic field	two readers	sweep value flags 85→2
3	the fix for 1 matched keyword arguments	authors	within 1
4	provider pin refused forced tool calls	authors	one battery 0/0 →usable
5	June pilot (11.5 %, sweep 1–100 %) did not reproduce; rolling alias, records not kept	authors	pilot not reported as a result
6	target harvest missed parameter defaults	second scorer	control 33→31
7	runner executed cells off the frozen plan	authors	48 runs relabeled
8	narration never recorded	authors	free text re-run, narration kept
9	profile floor stripped before scoring	its telemetry	`P-L1` 0→17
10	statistic names ours, not the profiler's	authors	$\approx\frac{1}{3}$ of first rate
11	cost table transcribed by hand	authors	table regenerated
12	magnitude aliases scored as signed	review	sweep value flags 2→1
13	correct class-balance claims called invented	re-run	`P-L1` wrong-number 17→9
14	free-text scorer: prompt names, wrong E for crowded, thousands separators	re-run	prompt arms 91–98→6–22 %
15	a verdict citing nothing not counted as committing	second scorer	stale-arm omissions 132→139
16	the word `target` taken for the frame's target column	review	invented 12→13; sweep 2→20
17	numbers of E read from the dump, without the derived absolute correlations	a judge's reasons	sweep extrinsic 34→21; audit items 15 of 17

11 Discussion

Cheap fixes and their reach Table 16 collects the conditions under which the failure appears and disappears. The misfiling of Section 6 disappears with the shipped prompt, with or without the prompt's claim that violations are rejected, or with one sentence of field description. The prompt's rules alone leave 7.0 %, and temperature 0 leaves 2.0 %. On another provider, the failure is absent altogether. Each fix stops working where the fix is absent: the failure reads 61 % on a differently named instance, 20–56 % on the fourteen public-data instances, and 49 % on the same model nine days later. The wrong numbers of Section 9 were absent under distinct column names, but we have not isolated naming as their cause. A tool author controls the prompt and the schema, but not the names in the user's data, the provider behind an endpoint, or the date. Without measuring each combination, the author cannot know that every field is specified well enough for every model. The effect of a fix is known only where the fix was measured, whereas the verifier's effect follows from code that runs in the caller's process.

Table 16: Where the failure appears: responses whose cited columns or claims break the contract, `deepseek-chat` with no verifier unless named, on the reference instance unless named. Rates are per 200 responses (per 20 for `qwen2.5:7b`).

Condition	Violating, %
Bare instruction, open schema, $T = 1.0$ (four runs)	40.5–49.0
the same with one sentence of field description	0.5
the same at $T = 0$	2.0
the shipped prompt's first three rules	7.0
the full shipped prompt, with or without its claims	0
a typed schema without a column list	4.0–7.0
an enum bound to E at call time	0
Bare instruction on the crowded instance (2026-09-24; 2026-10-02)	61.0; 57.0
an author-time enum covering 0 of 10 columns (A14)	71.5
the same covering 3, 5, or 8 of 10, anchor included	0–1.5
Bare instruction on twelve public-table instances	26.5–55.5
Bare instruction on two real datasets	20.5–21.5
Bare instruction, `gpt-5.4-mini`	0
Bare instruction, `qwen2.5:7b`	0

Binding at call time An enum written for the reference task turned 61 % misfiling into 72.5 % invention on the crowded instance, none of whose columns the enum lists; the call-time enum read zero. That control is extreme. Lists that kept the anchor and covered three to eight of the ten columns read 0 to 1.5 % invention, which 200 responses per list cannot tell apart from the list built at call time (A14). An author-time list is thus safe only as far as the list happens to cover the evidence, which the list's author cannot check in advance; binding at call time removes that dependence at no cost.

What carries over Four design lessons follow from the measurements and do not depend on mlcompass. First, admissibility should be indexed by field. A membership test over every name in the evidence passes the dominant failure: 98.7 % of the 1,831 violating responses on the bare leakage arms use a name that the evidence contains, in a field that does not admit that name. Second, the admissible sets should be derived from the producer's output at call time: a list written for other data turned misfiling into invention, whereas the list built at call time read zero, and a list written in advance protects only as far as the list happens to cover the evidence. Third, describing each field is the cheapest fix, and the effect of any fix of this kind holds only for the configuration in which the fix was measured (Table 16). Fourth, an evaluation of a narrator should report contract violations apart from what the violations introduce; counted as one hallucination rate, the failing model's errors would overstate invention many times over (Table 8). A model judge does not replace that check. Two inference models and two of the three LLM judges failed a 90 % bar on audit

items whose truth is fixed, and the judge that passed cost about two cents per item (Section 6), against microseconds for the check. The lessons apply to any narrator of a deterministic tool whose output is finite at call time.

Simpler designs If every structured fact were rendered deterministically and the model wrote only prose, the structured guarantee would be trivial, and the model would be left with the one channel that no check reaches. Stripping without a retry is cheaper, but stripping alone would have deleted content from 6–16 % of `A-STRESS` responses and from 41.5–59 % of the Guardrails arms' responses, all of which a single named retry repaired (Table 11). Applied offline to every unverified leakage response that violates the contract, deletion would keep 77.0 % of the claims and leave 554 responses with no claim at all, although nearly every deleted claim quotes a number that E carries somewhere (Appendix A.5); the retry is what avoids that loss. A further design lets the model choose evidence identifiers and has the host fill in every number. That design removes the value channel by construction, and the contract then reduces to Tier A with host rendering: the admissible sets are the same, bound at call time from the producer's output. Two of our results bear on the choice. The entity fields still need a check, because where the provider does not enforce the schema, as in every arm here, an enum is a request: 5 of 200 natural-name `P-TIER-A-ONLY` responses put a name outside the column domain into a column field. And a narrator whose output crosses the tool-server boundary, as the client model's does in the user's editor, cannot have its numbers filled in by our host; the tool can check and send back an answer that the client submits through a tool call, but cannot delete what the client tells the user, so only the retry transfers, not the guarantee, and our tool-server path runs without the contract. Where the host renders the answer, filling numbers by identifier is preferable for the value channel, and the verifier remains for the entity fields and the anchor.

Where the contract transposes The contract needs a deterministic producer, a finite set of observed entities and values, and a runtime domain bound to that set with independent verification. Narrators of static-analysis findings, continuous-integration failures, or dependency-scanner reports meet these conditions, and there, too, the guarantee would be consistency with the tool's output, not correctness.

12 Threats to Validity

Construct The guaranteed property is referential and value soundness on the structured channels. The verdict, the prose, and the correctness of E lie outside the property (Section 4). We measure faithfulness to E, not the effect of the narration on a reader's decisions. The outcome is contract compliance, which is not always a false statement. The two tasks treated dataset-level quantities differently: the leakage measurement accepts one under any listed column, whereas the profile contract offers no slot for one. Table 17 therefore re-scores every response under one policy for both tasks, strict or lenient. The leakage findings do not move: 98.7 % of the violating bare responses misfile under all three policies. The profile counts depend on the policy: the lenient one lowers the unverified natural-name count from 36 to 19 and the suffixed one from 29 to 22, and the wrong numbers remain.

Table 17: Responses that violate the contract under three policies for dataset- and task-level quantities. *Reported*: as in the other tables (leakage lenient, profile strict). *Strict*: no such quantity is admissible under a column, in either task. *Lenient*: both tasks admit one under any admissible column (on the profile task, a class share or the frame's row or column count, matched by name and value). Generated by `scripts/policy_sensitivity.py`.

Responses	N	Reported	Strict	Lenient
Leakage, bare instruction, four instances	3,200	1,014	1,014	1,014
Leakage, bare instruction, twelve instances	1,888	817	817	817
Leakage, six instruction wordings	600	370	374	370
Leakage, field described or shipped prompt	2,650	16	18	16
Profile, suffixed names, no verifier	600	29	29	22
Profile, natural names, no verifier	693	36	36	19
`gpt-5.4-mini`, every arm	1,450	3	3	0

The kinds of Table 8 separate what a response introduces from where the response puts an item, but a program assigns the kinds, and the kinds do not measure what a reader understands. The registered human audit (A7) was not carried out; the sheet is kept ready for independent raters. Two of its three automatic substitutes failed their validity checks. The one that passed (A13) gives a single model's classification, which agrees with ours on misfiled names and classifies them as false relations; the classification is not a person's, and the judge comes from the family of one narrator. The leakage conclusions hold under either reading, whereas the profile rates depend on the binding. The guarantee covers specified structured fields of one narration path, relative to trusted evidence; the guarantee does not show that the assistant explains leakage reliably.

The free text could in principle absorb what enforcement removes from the structured fields. A scan of 7,198 narrations, every number against every quantity in E and every identifier against every name in E and the prompt, found no clear increase: on the reference task, arms that can reject a claim carry ungrounded content in 8.8 % of narrations and arms that cannot in 12.6 %, and retried responses lean the way that displacement would predict by 2.5 and 3.4 points, which we cannot tell from noise (Appendix A.3). The scan cannot see misfiling displaced into the narration, because a misfiled name is in E. No enforced arm stripped anything, so the pressure we tested is correction rather than deletion. The value channel was stressed less than the entity channel: the registered value-pressure instance (H14) did not activate the value channel, and (C2) caught real errors only on the profile task.

Internal We compare arms within one date where we can. The field description ran on the same day as its floor (2026-09-24); the neutral description, temperature 0, and the prompt without its claim ran one to three days later against that floor.

Their differences (49 % against 0–2 %) are far larger than the floor's movement between runs (40.5–49 %), but these comparisons are not same-day controls. Arms ran in blocks, except in the registered runs A9.1, A11, and A14, which interleave their arms index by index, so drift within a day is controlled only there. The provider serves a rolling model name, and the unconstrained rates and retry counts move between runs, by more than the date alone explains (Appendix A.5). The harness and adapter also changed between runs, and the provenance of every record pins both, so a difference across dates cannot be attributed to the provider alone. The two naming schemes of the profile frame are such a difference. The registered interleaved run (A9.1) removes the date and harness from that comparison but did not reach significance (7 against 1 in 200, $p = 0.068$), so the naming explanation rests on which column each wrong number matches and remains exploratory.

External The misfiling occurs on one of three providers, so every comparative claim rests on a single provider. Every rate is conditioned on `deepseek-chat`, on the temperature (the floor reads 49.0 % at $T = 1.0$ and 2.0 % at $T = 0$), and on the date. Two leakage instances are synthetic, two use real datasets (one with a documented leak and one without), and twelve plant registered leaks in four public tables. On those twelve, the floor arm ran with 512 of its 2,400 calls lost, and the strong pair stopped early. The profile frame is synthetic data under two naming schemes. We registered a measurement of the shipped default narrator, `claude-opus-4-7`, and did not run the measurement for lack of budget. On the two real datasets, we called the narrator directly, because neither model score reaches the product's trigger, so the study evaluates the narration component rather than the product's default end-to-end use. We did measure the schema's cost: from 10 to 1,000 columns, the enum costs 38 bytes per column, against 344 for the evidence that produced the enum. Provider limits on enum size are unmeasured, and the retry, which inlines the list, grows with the domain.

Conclusion validity Two hundred responses to one evidence dictionary measure one configuration's conditional rate. These responses are not 200 tasks, and the total of 19,132 responses is not the sample size of any comparison. The sixteen leakage instances come from eight sources: two synthetic frames, two real datasets, and four public tables carrying twelve injected leaks. At $N = 200$, a comparison against a zero arm reaches $p < 0.05$ at six events and detects a true rate of 4 % with power 0.8 (Fisher), but detects a true rate of 2 % with power of only 0.21. The profile difference of 4 in 200 was therefore out of reach of the run that found the difference. The registered test A11 used 600 responses per arm, where six events against a zero arm reach $p < 0.05$ and a true rate of 1.5 % is detected with power 0.89. The resolution is coarser elsewhere: near the 42 % floor, two arms must differ by about 14 points for the same power (13.8 points downward and 14.4 upward; two-sided Fisher), so mechanisms closer than that cannot be separated there. Only the registered family is corrected for multiplicity (Table 2); every other interval is descriptive.

13 Conclusion

We asked how an LLM narrator of a deterministic tool's structured output fails, and what a check bound to that output can guarantee. On the one model of three that fails often under a bare instruction, misfiling rather than invention dominates the violations. Of the 1,829 flagged responses on sixteen leakage instances, 1,807 place an item that the evidence contains, above all the metric's name `r2`, in a field that does not admit the item, and 13 use an invented name. The other two models fail rarely: `qwen2.5:7b` in 2 of 20 responses under a typed schema without a column list, and `gpt-5.4-mini` by misplacing the class balance in 3 of 50 profile probes. The shipped prompt, a field description, or another provider removes the failure, temperature 0 nearly does, and the failure returns wherever the fix is absent. We claim no general rate of misfiling for LLMs, only that misfiling can dominate when an evidence-closed narrator fails, and that a single hallucination rate would hide the difference.

For narration whose citable entities and quantities are finite and enumerable at call time, binding each field's admissible set to the evidence and checking the returned payload in host code enforces referential and value soundness behind an API that exposes no logits. The mechanism is ordinary validate-and-reask. The check held in every configuration in which the check ran, repaired every violation with a named retry, at the first attempt in all but three responses, and costs microseconds; a reading that shares no code with the verifier confirmed every rejection that the records keep. In the best configuration that we found, a registered test measured what the check adds: 10 violating responses in 600 without the check and none in 600 with the check, with no observed reduction in the content measures. The one model judge that passed a registered validity check classified the dominant misfiling as a false relation; how people read the misplaced items remains open. We report seventeen faults in our own instruments and two in the verifier, with what each fault changed. The study evaluates the narration component: the shipped default narrator and the product's trigger path were not exercised end to end. The check does not yet reach a client model that narrates the same evidence across the tool-server boundary.

A Audit Trail

This appendix keeps the parts of the record that the main text summarizes: the registered items that the results do not rest on, the automatic judges that failed their validity check, the instrument faults in detail, the free-text scan, the verdicts, further results, and the downstream benchmark.

Table A1: The registered items that the results do not rest on, continuing Table 2.

Item	Prediction	Outcome	Status
A3 R3	Measure the shipped default narrator	Not run: no budget	not tested
A3 R4	Blind human audit of the scorer on 200 responses	Not run (A5); superseded by A7	not tested
A7	Blind semantic audit of 204 items in nine strata: most misfiled names read as right information in the wrong place (P1), most wrong numbers that carry another item's value read as false (P2), and at least 90 % of controls read as supported (P3)	Registered on 2026-10-02 and the sheet drawn; the authors did not label the sheet	not run
A10	Two NLI judges pass a validity check on the audit items whose truth is fixed (at least 90 % each way)	Neither passed: DeBERTa 55/55 supported and 60/79 not supported, RoBERTa 53/55 and 31/79, $\kappa = 0.25$; reported as uninformative	as planned
A12	Two LLM judges (`gpt-5.4-mini`; `deepseek-chat` after A12a) pass the same validity check	Neither passed: `gpt-5.4-mini` 41/55 not-false items as A or B and 66/79 false items as C or D, `deepseek-chat` 48/55 and 70/79, $\kappa = 0.63$; reported as uninformative	as planned
A9.2	Leakage retry audit, every attempt kept: the second scorer confirms at least 95 % of rejections	17 of 17 confirmed and 17 of 17 final attempts clean; 170 correct claims before and after the retries	as planned

Table A2: Every rejected attempt that the records keep (profile task, `deepseek-chat`), re-judged without the verifier. *Finding*: what an independent reading of E finds; for a wrong number, the column whose value the number is. *Retry*: what the next attempt did with the item. *Run*: *re-run* is the 2026-09-24 re-run of `P-CONTRACT` on the suffixed frame that kept every attempt (Table 14 reports the 2026-09-19 run, which kept none); *strong* is the strong natural-name run of Table 14, whose three content rejections and one verdict rejection are rows 77, 127, 140, and 81; *R1* is the registered confirmatory run that stopped early (A3); *A11* is the registered interleaved test. An attempt with more than three offending items of one finding takes one row. Every final attempt was clean. Generated by `scripts/rejudge_rejections.py`.

Run	Arm	i	Item	Finding	Retry
re-run	`P-CONTRACT`	2	`cat_feature_03.iqr_count`	pair not in E	dropped
re-run	`P-CONTRACT`	5	`num_feature_07.missing_pct`	value of `cat_feature_07`	corrected
re-run	`P-CONTRACT`	5	`num_feature_07.missing_count`	value of `cat_feature_07`	corrected
re-run	`P-CONTRACT`	63	`num_feature_06.missing_pct`	value of `cat_feature_06`	corrected
re-run	`P-CONTRACT`	105	`num_feature_07.missing_pct`	value of `cat_feature_07`	dropped
re-run	`P-CONTRACT`	108	`num_feature_07.missing_pct`	value of `cat_feature_07`	dropped
re-run	`P-CONTRACT`	120	`num_feature_07.missing_pct`	value of `cat_feature_07`	corrected
re-run	`P-CONTRACT`	181	`num_feature_07.missing_pct`	value of `cat_feature_07`	dropped
re-run	`P-CONTRACT`	194	`num_feature_07.missing_pct`	value of `cat_feature_07`	dropped
R1	`P-STRONG-CONTRACT`	6	`deprecated:discount_rate.z_score_count`	column not in E	dropped
strong	`P-STRONG-CONTRACT`	77	`referral_team`	column not in E	dropped
strong	`P-STRONG-CONTRACT`	77	`referral_team.missing_pct`	column not in E	dropped
strong	`P-STRONG-CONTRACT`	77	`referral_team.missing_count`	column not in E	dropped
strong	`P-STRONG-CONTRACT`	81	`verdict`	no admissible verdict	verdict supplied
strong	`P-STRONG-CONTRACT`	127	`reference_count`	column not in E	dropped
strong	`P-STRONG-CONTRACT`	140	`access_channel`	column not in E	dropped
A11	`P-STRONG-CONTRACT`	4	`risk_score`	column not in E	dropped
A11	`P-STRONG-CONTRACT`	77	`daystreak`	column not in E	dropped
A11	`P-STRONG-CONTRACT`	279	`agency.missing_pct`	column not in E	dropped
A11	`P-STRONG-CONTRACT`	279	`balance_placeholder_removed`	column not in E	dropped
A11	`P-STRONG-CONTRACT`	293	`device_type.capitality`	pair not in E	dropped
A11	`P-STRONG-CONTRACT`	475	`label`	column not in E	dropped
A11	`P-STRONG-CONTRACT`	480	`tenant_months`	column not in E	dropped
A11	`P-STRONG-CONTRACT`	481	`households`	column not in E	dropped
A11	`P-STRONG-CONTRACT`	490	`district`, `crim`, and 30 more	column not in E	dropped
A11	`P-STRONG-CONTRACT`	502	`months.missing_count`	column not in E	dropped
A11	`P-STRONG-CONTRACT`	568	`setting_minutes`	column not in E	dropped

A.1 Automatic judges that failed the validity check

The authors did not label the registered blind audit by hand (A7), so we registered an automatic substitute before running the substitute (amendment A10). Two natural-language-inference models, trained by others on human-annotated data (DeBERTa-v3-large on five inference datasets and RoBERTa-large on MultiNLI), judged whether E supports, contradicts, or does not address each of the 204 audit items, read as a statement. The registration required each judge to match the known truth on at least 90 % of the items whose truth is fixed by construction. Admissible names, matching claims, and real columns are supported; wrong numbers, invented names, and numbers that E records nowhere are not. Neither judge met that bar. DeBERTa labeled all 55 known-supported items supported but only 60 of the 79 known-unsupported items as not supported; RoBERTa reached 53 of 55 and 31 of 79; and the two judges agreed on 63 % of the items (Cohen's $\kappa = 0.25$). As registered, we do not interpret their labels on the contested strata, and the automatic audit is uninformative on these data. One pattern is worth recording as an exploratory observation. DeBERTa labeled all 22 wrong numbers contradicted, and 16 of its 19 misses on known-unsupported items are names close to a real one: twelve are the word `target`, and four are respellings such as `acquired_channel`. These are exactly the items on which the literal rule and a reader's reading part ways.

A second registered substitute (amendment A12) put the human sheet's question, with its instructions, to two LLMs from different families: `gpt-5.4-mini` and `deepseek-chat`, from the provider of the narrator that fails most often. DeepSeek replaced Mistral, which answered every call with a rate-limit error (A12a). The validity check was the same in kind: at least 90 % of the items that are not false labeled A or B, and at least 90 % of the false or absent items labeled C or D. Neither judge met that bar. `gpt-5.4-mini` reached 41 of 55 and 66 of 79, `deepseek-chat` 48 of 55 and 70 of 79, and the two judges agreed on 76 % of the items (Cohen's $\kappa = 0.63$). As registered, we do not interpret their labels on the contested strata. Their errors are still worth recording. `gpt-5.4-mini` labeled 11 of the 22 contradicted numbers as supported; each of the 11 quotes, as the claimed column's value, the number that the same-suffix sibling records for the same statistic. Both judges labeled the misfiled metric name `r2` as absent from E (40 and 36 of 40), although E lists the name and the instructions forbid that label. `gpt-5.4-mini` did the same for 14 of the 15 real columns that the frame's column list shows. The judges thus accepted a sibling's number, which the value check rejects, and read misfiling as invention, the conflation that Table 8 separates.

A.2 Instrument faults in detail

Fault 14 was caught before its numbers were reported and changed what the free-text measurement counts. Fault 15 was found by a second scorer, written from the definitions and sharing no code with the first scorer, which scored all 16,698 leakage records. The two scorers agree on every entity flag, misfiled flag, and out-of-evidence flag. They disagree on 10 wrong-number flags, each a derived quantity such as a correlation gap, which the first scorer rightly files as a pair that E does not carry. They also disagree on 7 responses on the stale-enum arm that commit to a verdict and cite nothing: the definition counts these responses as omitting the anchor, and the first scorer did not. Both scorers come from the same definitions, so their agreement speaks to the implementations, not to the definitions. Fault 16 was found in review: two passages called the word `target` a real column of the frame, although the frame's target column is `y_true`. By our rule, the word is invented, which moves the invented names on the bare leakage arms from 12 to 13 of 1,829 and on the wording sweep from 2 to 20. Fault 17 was found in the reasons that the A13 judge gave. The kinds and the audit sheet read "a number that E carries" as a number of the evidence dump, whereas the value table that the contract binds, and that the sheet shows, also derives each correlation's absolute value. Counted with the table, 13 responses of the wording sweep move from extrinsic to misplaced, one bare response moves from extrinsic to contradicting E, and 15 of the 17 audit items drawn as numbers that E records nowhere carry a number that the sheet shows. The sheet keeps its registered draw, and every audit score now reports the corrected strata beside the drawn ones.

Three of the faults show patterns that the others repeat. In faults 2, 12, and 13, a scorer reported the narrator wrong for being right. The scorer compared every claim with its column's correlation whatever statistic the claim named, compared `abs_corr` with a signed value, and called a class balance invented although the profiler reports one. We found fault 2 because two readers disagreed about what a channel measured. Fault 9 is a control that silently received the treatment. The profile floor ran the deletion step unconditionally and read 0 % on a task that fails, and we noticed only because the arm printed sixty Tier B catches although the arm has no Tier B. The corrections also had a direction. The first four corrections that moved the downstream checklist all moved the checklist toward our hypothesis, because false positives concentrate in careful code and the treatment arms write more of that code. The two corrections that broke the pattern came from procedures designed to look for such faults. One was a second scorer written with a different technique, which agreed with the first scorer on 856 of 858 rule instances before fault 6 was fixed and on all 858 afterwards. The other was an arm whose adverse outcome we wrote down before the arm ran.

A.3 The free-text scan

The free text could in principle absorb what enforcement removes from the structured fields. Every run on the two synthetic instances since the record normalizer was fixed (fault 8) keeps the narration: 7,198 narrations on `deepseek-chat` in 36 cells. We checked every number in each narration against every quantity in E, and every identifier against every name in E and in the system prompt. On the reference task, arms that

can reject a claim carry ungrounded content in 158 of 1,800 narrations (8.8 % [7.6, 10.2]), and arms that cannot reject a claim carry such content in 378 of 3,000 (12.6 % [11.5, 13.8]). Within the rejecting arms, responses that went through a retry read 28/256 (10.9 % [7.7, 15.4]) against 130/1,544 (8.4 % [7.1, 9.9]) for the rest (Fisher $p = 0.19$). On the crowded instance, the rejecting arms read 71/1,200 (5.9 % [4.7, 7.4]) and the others 45/999 (4.5 % [3.4, 6.0]), leaving aside the stale-enum arm, whose narrations repeat the invented names of its structured fields (117/199, 58.8 %). There, retried responses read 21/244 (8.6 %) against 50/956 (5.2 %) ($p = 0.07$). The retried responses lean the way that displacement would predict, by 2.5 and 3.4 points, which we cannot tell from noise at this sample size. We therefore claim only that the scan found no clear increase, within what the scan can see. The instrument has one blind spot: a misfiled name is in E and so counts as grounded in prose, so misfiling displaced into the narration would not register. The rejected attempts of the main leakage runs were not kept, so those rejections cannot be re-judged; the registered re-run of amendment A9.2 kept every attempt, and the second scorer confirmed all 17 of its rejections. No enforced arm stripped anything, which means that the pressure we tested is correction rather than deletion, and unsupported causal prose would need human raters. Run at call time as a warning, the same matcher would fire on 5–13 % of narrations, mostly on true bounds such as "below 0.05" and on shortened names; mlcompass does not ship the matcher. The value channel was stressed less than the entity channel: the registered value-pressure instance (H14) did not activate the value channel, and (C2) caught real errors only on the profile task.

A.4 Verdicts

No response cited the planted leak and answered "score legitimate". Of the 16,698 leakage responses, 9,162 carry one of the four labels; the other 7,536 come from open-schema arms, whose verdict is free text (7,523) or empty (13). Of the 8,363 labeled responses on instances with a leak, planted or real, 8,263 say "leakage likely", 54 say "leakage uncertain", and 46 abstain. On the negative control, none of the 799 responses says "leakage likely": 381 abstain, as the shipped prompt asks when there is no candidate, 238 say "uncertain", and 180 say "score legitimate". These counts are observations, not a property of the contract.

A.5 Further results

Results in neither table `A-L2-DESCRIBED` and `A-L2-DESCRIBED-NEUTRAL` read 0/200; the shipped contract under the RQ4 wording that fails most often returned no violation; and on the crowded instance, `A-STRESS`, `A-GR-CHOICES`, and `A-GR-OURS` retried 25, 101, and 118 of 200 responses. The contract path does not surface token counts.

Drift between runs `A-GR-STOCK` read 35.5, 43.5, and 47.5 % over nine days (heterogeneity $p = 0.047$), and `A-L1` did not move ($p = 0.34$). `A-STRESS` repaired 12, 23, 32, and 19 responses in four runs ($p = 0.012$), with 32 against 19 on the same date ($p = 0.07$), so the date does not account for all of the variation.

Deletion without a retry Applied offline to the 2,089 violating responses of every unverified leakage arm, deletion would keep 87.5 % of their cited columns and 77.0 % of their claims, would never lose the anchor, and would leave 554 responses with no claim. Nearly every claim that deletion removes (3,865 of 3,888) quotes a number that E carries somewhere, mostly under a statistic name that the domain does not admit. That fact does not make each such claim correct, but deletion would discard all of them, and the retry is what avoids the loss.

A.6 A downstream benchmark

A pre-registered benchmark of whether the tool's findings improve the training scripts that a model writes is in the replication package with its protocol, runs, and both scorers. The benchmark measures the tool's advice, not the narration contract, and the benchmark cannot show a benefit even of the advice. A control arm told only the six scored defects ended with fewer of those defects than the arm given the tool's findings, while producing a runnable script in 47 % of runs against 78–83 % for the other arms. We draw no claim from the benchmark.

Acknowledgments

We thank the readers of earlier drafts whose disagreement located the value scorer's defect.

Declaration of AI-assisted tools The authors used the artificial intelligence (AI) assistant Claude (Anthropic), principally Claude Opus 5.5, accessed through Claude Code during 2026. Claude drafted and revised manuscript text in every section from the authors' notes, results and instructions, including a final editing pass for readability; wrote and tested most of the measurement harness, the analysis scripts that compute the reported tables, and parts of the artifact's verification code; ran the experiment batteries under the authors' direction; checked bibliographic entries against their sources; simulated a multi-reviewer pre-submission review whose findings the authors addressed; built the sheet and the sampling script of the registered blind audit, which the authors decided not to label; and wrote the scripts through which model judges labeled the audit items in its place. The large language models `deepseek-chat`, `gpt-5.4-mini` and `qwen2.5:7b` are subjects of the study and are described in Section 5; two natural language inference models and the LLMs `gpt-5.4-mini`, `deepseek-chat` and `gpt-5.5` served as judges of the semantic audit (Section 6). The authors chose the research questions and the study design, reviewed and edited all text, inspected and tested the code, recomputed every reported number from the released run records, verified every reference, and made

all analytical and interpretive decisions. Claude is not an author, and the authors take full responsibility for the content of this paper.

Data Availability

The replication package (Sabuniş et al., 2026) contains the artifact source, every harness, and the frozen analysis plan with its amendments. The package also holds one record per live response with the verbatim reply delivered to the user, the scripts that recompute every table from those records, and an inventory of every run file from which each count in this paper is recomputed. The registered property-based falsifier, the superseded measurements beside their corrections, the blind-audit sheet with its sampling script, and the downstream benchmark complete the package. Profile records written from 2026-09-24 on keep every attempt. The earlier profile records and all reported leakage records keep the delivered payload with the verifier's verdict on each attempt (the count and kind of each rejection) but not the rejected payloads, so those rejections cannot be re-judged independently. The leakage harness has kept every attempt since 2026-10-01, and the registered re-run of 2026-10-02 (amendment A9.2) used that version. Code is released under the MIT license and data under CC-BY 4.0. Most of the harness and analysis code was drafted with an AI assistant and then inspected and tested by the authors.

Statements and Declarations

Funding. No funding was received for conducting this study.

Competing interests. The authors have no competing interests to declare that are relevant to the content of this article. mlcompass is released by the authors under the MIT license and is not a commercial product.

Ethics approval. Not applicable: the study involved no human participants and no animals. The registered blind audit was not labeled by people; model judges labeled its items, as the paper reports.

Consent to participate and to publish. Not applicable.

Author contributions. H.S. designed and implemented the contract and the measurement harnesses, and ran the batteries. Y.Ü. designed the downstream benchmark protocol and performed the independent review that identified scorer false positives. M.K.Ö. supervised the work. All authors read and approved the final manuscript.

Code availability. The source of mlcompass, every harness and every analysis script are in the replication package (Sabuniş et al., 2026) under the MIT license; the development repository is `https://github.com/hakansabunis/mlcompass`.

References

Martín Abadi, Paul Barham, Jianmin Chen, Zhifeng Chen, Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Geoffrey Irving, Michael Isard, Manjunath Kudlur, Josh Levenberg, Rajat Monga, Sherry Moore, Derek G. Murray, Benoit Steiner, Paul Tucker, Vijay Vasudevan, Pete Warden, Martin Wicke, Yuan Yu, and Xiaoqiang Zheng. TensorFlow: A system for large-scale machine learning. In *Proceedings of the 12th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 265–283, 2016. doi: 10.48550/arXiv.1605.08695.

Lakshya A. Agrawal, Aditya Kanade, Navin Goyal, Shuvendu K. Lahiri, and Sriram K. Rajamani. Monitor-guided decoding of code LMs with static analysis of repository context. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. doi: 10.52202/075280-1401.

Joongho Ahn and Moonsoo Kim. From prompts to contracts: Harness engineering for auditable enterprise LLM agents. arXiv:2607.08028, 2026. arXiv preprint; accessed 2026-09-18.

Anthropic. Model context protocol specification. `https://modelcontextprotocol.io`, 2024.

Luca Beurer-Kellner, Marc Fischer, and Martin Vechev. Prompting is programming: A query language for large language models. *Proceedings of the ACM on Programming Languages*, 7(PLDI):1946–1969, 2023. doi: 10.1145/3591300.

Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, et al. Evaluating large language models trained on code. arXiv:2107.03374, 2021.

Ruiying Chen. Evidence-bound autonomous research (EviBound): A governance framework for eliminating false claims. arXiv:2511.05524, 2025.

Christopher Cruz. ATLAS-RTC: Closing the loop on LLM agent output with token-level runtime control. arXiv:2603.27905, 2026. arXiv preprint; accessed 2026-09-18.

Dean De Cock. Ames, Iowa: Alternative to the Boston housing data as an end of semester regression project. *Journal of Statistics Education*, 19(3), 2011. doi: 10.1080/10691898.2011.11889627.

Bhuwan Dhingra, Manaal Faruqui, Ankur Parikh, Ming-Wei Chang, Dipanjan Das, and William Cohen. Handling divergent reference texts when evaluating table-to-text generation. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2019. doi: 10.18653/v1/P19-1483.

Enjun Du, Hange Zhou, Chenxu Du, Siyi Liu, Zirong Chen, Ziyu Zheng, and Yongqi Zhang. LedgerMind: Provenance-constrained multimodal agentic reasoning with a structured evidence ledger. arXiv:2607.28374, 2026. arXiv preprint; accessed 2026-09-18.

Ondřej Dušek, Jekaterina Novikova, and Verena Rieser. Evaluating the state-of-the-art of end-to-end natural language generation: The E2E NLG challenge. *Computer Speech & Language*, 59:123–156, 2020. doi: 10.1016/j.csl.2019.06.009.

Yliès Falcone, Jean-Claude Fernandez, and Laurent Mounier. What can you verify and enforce at runtime? *International Journal on Software Tools for Technology Transfer*, 14(3): 349–382, 2012. doi: 10.1007/s10009-011-0196-8.

Matthias Feurer, Aaron Klein, Katharina Eggensperger, Jost Tobias Springenberg, Manuel Blum, and Frank Hutter. Efficient and robust automated machine learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, pages 2962–2970, 2015. URL `https://papers.nips.cc/paper/5872-efficient-and-robust-automated-machine-learning.pdf`.

Saibo Geng, Martin Josifoski, Maxime Peyrard, and Robert West. Grammar-constrained decoding for structured NLP tasks without finetuning. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 10932–10952, 2023. doi: 10.18653/v1/2023.emnlp-main.674.

Saibo Geng, Hudson Cooper, Michał Moskal, Samuel Jenkins, Julian Berman, Nathan Ranchin, Robert West, Eric Horvitz, and Harsha Nori. JSONSchemaBench: A rigorous benchmark of structured outputs for language models. arXiv:2501.10868, 2025. arXiv preprint; accessed 2026-09-27.

Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec)*, pages 79–90, 2023. doi: 10.1145/3605764.3623985.

Guardrails AI. Guardrails AI: Adding guardrails to large language models. `https://github.com/guardrails-ai/guardrails`, 2026. Software repository. Accessed 14 September 2026.

Lei Huang, Weijiang Yu, Weitao Ma, Weihong Zhong, Zhangyin Feng, Haotian Wang, Qianglong Chen, Weihua Peng, Xiaocheng Feng, Bing Qin, and Ting Liu. A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions. arXiv:2311.05232, 2023.

IBM. Telco customer churn (sample data set). `https://github.com/IBM/telco-customer-churn-on-icp4d`, 2019.

Andras Janosi, William Steinbrunn, Matthias Pfisterer, and Robert Detrano. Heart disease. UCI Machine Learning Repository, 1988.

Ziwei Ji, Nayeon Lee, Rita Frieske, Tiezheng Yu, Dan Su, Yan Xu, Etsuko Ishii, Ye Jin Bang, Andrea Madotto, and Pascale Fung. Survey of hallucination in natural language generation. *ACM Computing Surveys*, 55(12):1–38, 2023. doi: 10.1145/3571730.

Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations (ICLR)*, 2024. doi: 10.48550/arXiv.2310.06770.

Nimisha Karnatak, Mohamad Chatila, Daniel Alejandro Pinzón Hernández, Reza Yazdanfar, Michelle Dugas, and Renos Vakis. Learning from AVA: Early lessons from a curated and trustworthy generative AI for policy and development research. arXiv:2604.17843, 2026. arXiv preprint; accessed 2026-09-27.

Shachar Kaufman, Saharon Rosset, Claudia Perlich, and Ori Stitelman. Leakage in data mining: Formulation, detection, and avoidance. *ACM Transactions on Knowledge Discovery from Data*, 6(4):1–21, 2012. doi: 10.1145/2382577.2382579.

Brett Lantz. *Machine Learning with R*. Packt Publishing, Birmingham, UK, 2013.

Yifan Le. Schema-key wording as an instruction channel in structured generation under constrained decoding. arXiv:2604.14862, 2026. arXiv preprint; accessed 2026-09-18.

Yin Li. When JSON is not enough: Semantic reliability of schema-constrained LLM ordering agents. arXiv:2607.18261, 2026. arXiv preprint; accessed 2026-09-28.

Jay Ligatti, Lujo Bauer, and David Walker. Edit automata: Enforcement mechanisms for run-time security policies. *International Journal of Information Security*, 4(1–2):2–16, 2005. doi: 10.1007/s10207-004-0046-8.

Jason Liu. Instructor: Structured outputs for LLMs. `https://github.com/instructor-ai/instructor`, 2023. Software; validation context and re-asking on failure.

Scott Lundberg and Marco Túlio Ribeiro. Guidance: A guidance language for controlling large language models. https://github.com/guidance-ai/guidance, 2023.

Qinglan Luo, S M A Nahian, John Guttag, S. Mazdak Abulnaga, and Katie Matton. A removal based approach to improve LLM faithfulness at test-time. arXiv:2609.04343, September 2026.

Joshua Maynez, Shashi Narayan, Bernd Bohnet, and Ryan McDonald. On faithfulness and factuality in abstractive summarization. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 1906–1919, 2020. doi: 10.18653/v1/2020.acl-main.173.

Bertrand Meyer. Applying "design by contract". *Computer*, 25(10):40–51, 1992. doi: 10.1109/2.161279.

David Muchlinski, David Siroky, Jingrui He, and Matthew Kocher. Comparing random forest with logistic regression for predicting class-imbalanced civil war onset data. *Political Analysis*, 24(1):87–103, 2016. doi: 10.1093/pan/mpv024.

Albus W. Ng, Yi Han, Jusheng Zhang, and Wenhao Wang. Agent safety should be a runtime contract. arXiv:2608.11274, August 2026.

OpenAI. Structured outputs. https://platform.openai.com/docs/guides/structured-outputs, 2024.

K. W. Penrose, A. G. Nelson, and A. G. Fisher. Generalized body composition prediction equation for men using simple measurement techniques. *Medicine and Science in Sports and Exercise*, 17(2):189, 1985. doi: 10.1249/00005768-198504000-00037.

Gabriel Poesia, Oleksandr Polozov, Vu Le, Ashish Tiwari, Gustavo Soares, Christopher Meek, and Sumit Gulwani. Synchromesh: Reliable code generation from pre-trained language models. In *International Conference on Learning Representations (ICLR)*, 2022.

Paul Ralph et al. ACM SIGSOFT empirical standards for software engineering research. arXiv:2010.03525, 2020.

Traian Rebedea, Razvan Dinu, Makesh Narsimhan Sreedhar, Christopher Parisien, and Jonathan Cohen. NeMo Guardrails: A toolkit for controllable and safe LLM applications with programmable rails. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP): System Demonstrations*, pages 431–445, 2023. doi: 10.18653/v1/2023.emnlp-demo.40.

Raymond Reiter. On closed world data bases. In Hervé Gallaire and Jack Minker, editors, *Logic and Data Bases*, pages 55–76. Plenum Press, 1978. doi: 10.1007/978-1-4684-3384-5_3.

Hakan Sabuniş, Yusuf Ünlü, and Mehmet Kemal Özdemir. mlcompass: Artifact and replication package. Zenodo, all versions, 2026. Source code: https://github.com/hakansabunis/mlcompass.

Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. doi: 10.48550/arXiv.2302.04761.

Fred B. Schneider. Enforceable security policies. *ACM Transactions on Information and System Security*, 3(1):30–50, 2000. doi: 10.1145/353323.353382.

Torsten Scholak, Nathan Schucher, and Dzmitry Bahdanau. PICARD: Parsing incrementally for constrained auto-regressive decoding from language models. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 9895–9901, 2021. doi: 10.18653/v1/2021.emnlp-main.779.

Melanie Sclar, Yejin Choi, Yulia Tsvetkov, and Alane Suhr. Quantifying language models' sensitivity to spurious features in prompt design. In *International Conference on Learning Representations (ICLR)*, 2024. doi: 10.48550/arXiv.2310.11324.

Quentin Signé, Mohand Boughanem, Jose Moreno, and Thiziri Belkacem. Guaranteeing faithful evidence extraction in speculative retrieval-augmented generation. arXiv:2609.10046, September 2026.

Owen Truong, Terrence Zhang, Arnav Marchareddy, Ryan Lee, Jeffery Busold, Michael Socas, and Eman Abdullah AlOmar. LeakageDetector 2.0: Analyzing data leakage in jupyter-driven machine learning pipelines. arXiv:2509.15971, September 2025.

Yiqi Wang, Jiaqi Zhang, Zhangkai Wu, Taotao Cai, Zirui Liu, Zequn Sun, Manqing Dong, Mingkai Zheng, Yiqun Duan, Xuefei Yin, and Yanming Zhu. From agent traces to trust: A survey of evidence tracing and execution provenance in LLM agents. arXiv:2606.04990, June 2026.

Brandon T. Willard and Rémi Louf. Efficient guided generation for large language models. arXiv:2307.09702, 2023.

Cailin Winston, Claris Winston, and René Just. Solver-aided verification of policy compliance in tool-augmented LLM agents. arXiv:2603.20449, 2026. arXiv preprint; accessed 2026-09-27.

Sam Wiseman, Stuart Shieber, and Alexander Rush. Challenges in data-to-document generation. In *Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 2253–2263, 2017. doi: 10.18653/v1/d17-1239.

Bowen Xu. Hallucination is inevitable for LLMs with the open world assumption. arXiv:2510.05116, September 2025.

Chenyang Yang, Rachel A. Brower-Sinning, Grace A. Lewis, and Christian Kästner. Data leakage in notebooks: Static detection and better processes. In *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2022. doi: 10.1145/3551349.3556918.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023. doi: 10.48550/arXiv.2210.03629.

YData. ydata-profiling: Automated exploratory data analysis for pandas dataframes. `https://github.com/ydataai/ydata-profiling`, 2026. Software repository. Accessed 14 September 2026.

Matei Zaharia, Andrew Chen, Aaron Davidson, Ali Ghodsi, Sue Ann Hong, Andy Konwinski, Siddharth Murching, Tomas Nykodym, Paul Ogilvie, Mani Parkhe, Fen Xie, and Corey Zumar. Accelerating the machine learning lifecycle with MLflow. *IEEE Data Engineering Bulletin*, 41(4):39–45, 2018. URL `https://dblp.org/db/journals/debu/debu41.html`.

Shuoming Zhang, Jiacheng Zhao, Hanyuan Dong, Ruiyuan Xu, Zhicheng Li, Yangyu Zhang, Shuaijiang Li, Yuan Wen, Chunwei Xia, Zheng Wang, Xiaobing Feng, and Huimin Cui. When grammar guides the attack: Uncovering control-plane vulnerabilities in LLMs with structured output. arXiv:2503.24191, 2025a. arXiv preprint; accessed 2026-09-28.

Yedi Zhang, Sun Yi Emma, Annabelle Lee Jia En, and Jin Song Dong. RvLLM: LLM runtime verification with domain knowledge. arXiv:2505.18585, 2025b. arXiv preprint; accessed 2026-09-27.